AI Voice Sales & Customer Lifecycle Agent

AI Voice Sales & Customer Lifecycle Agent

Solution Overview & Engineering Approach

---

1. Executive Summary

We are building an autonomous AI voice agent that manages the complete customer lifecycle over telephone calls: outbound sales outreach, inbound query handling, appointment booking and management, customer support, and follow-up engagement.

The system conducts natural voice conversations, maintains persistent memory of every customer interaction, retrieves answers from the company's official knowledge base, and executes business actions (bookings, tickets, CRM updates) — all while operating strictly within configurable business rules, with a complete audit trail of every decision.

Three principles define the engineering approach:

1. **Separation of reasoning and execution.** The language model proposes actions; deterministic application code validates and executes them. The model has no direct ability to modify data or place bookings.
2. **Platform-generic design.** The engine contains no industry-specific logic. Business rules, customer journey stages, tools, and conversational persona are loaded as configuration ("industry packs"), making the platform reusable across clients and domains without code changes.
3. **Auditability by construction.** Every decision is recorded with its full context — inputs, retrieved data, rules evaluated, confidence, and outcome — enabling the system to explain any action after the fact.

---

2. Capabilities

Function	Capabilities
Sales	Outbound sales calls, inbound call handling, product Q&A grounded in company documents, objection handling, lead qualification
Appointments	Real-time slot search, booking, rescheduling, cancellation, reminder calls, no-show follow-up — with capacity, working hours, holidays, and buffer times enforced

Support	Ticket creation, grievance handling, escalation to human agents with full context transfer
Lifecycle	Persistent per-customer memory, journey stage tracking, retry and callback management, campaign-based outreach
Governance	Do-Not-Contact enforcement, calling-hour compliance, consent tracking, recording disclosure, decision-level audit logs

---

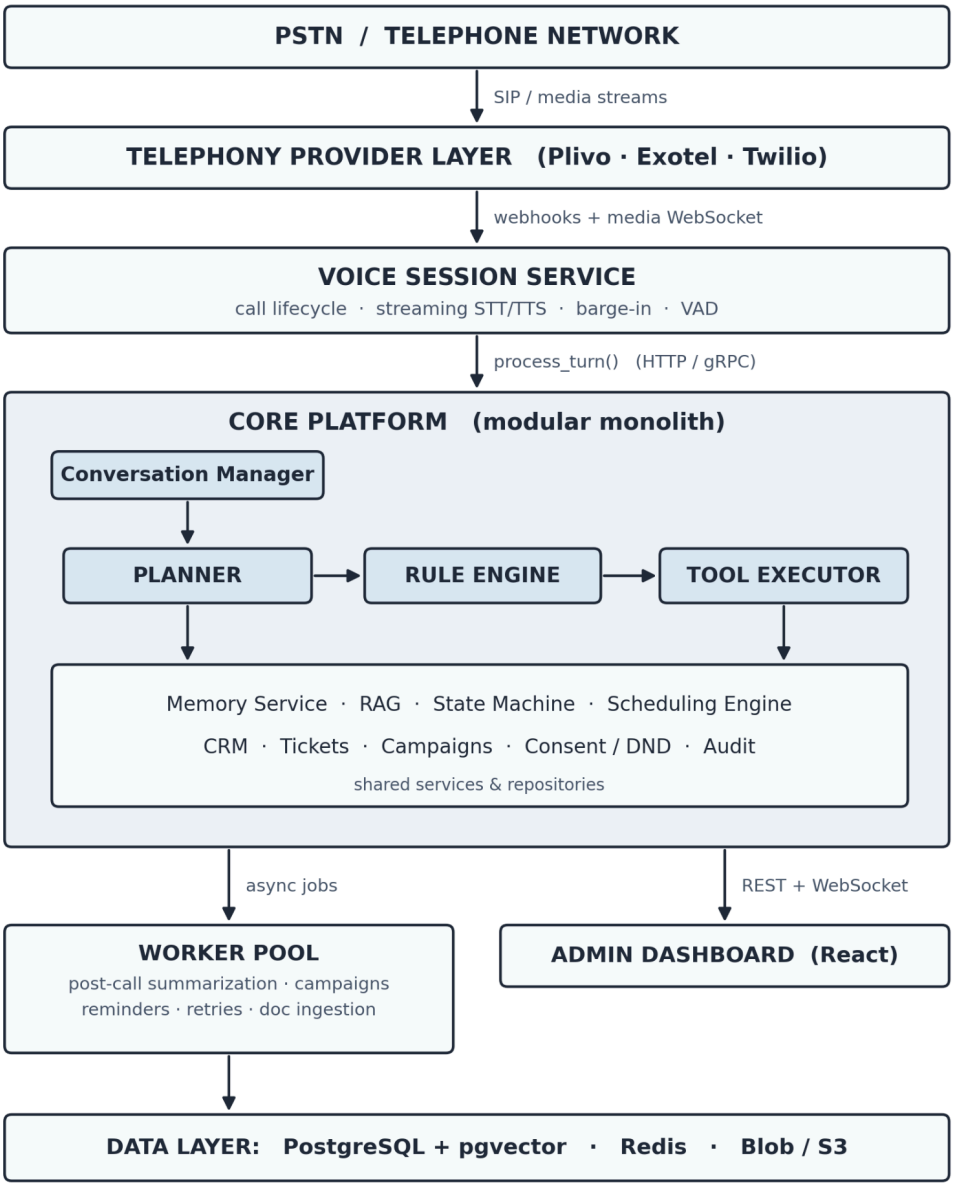
3. Technology Stack

Layer	Technology	Rationale
Backend framework	Python 3.12, FastAPI (async)	Team's established stack; strong async support required for concurrent call handling
Database	PostgreSQL 16 with pgvector	Single store for relational data (CRM, scheduling, audit) and vector embeddings (knowledge retrieval); avoids operating a separate vector database
Cache / queues	Redis 7	Session state for live calls, task queues, distributed locks for booking concurrency
Background jobs	Celery workers	Post-call processing, campaign dialing, reminders, retries, document ingestion
Frontend	React 18, Vite, Tailwind CSS	Admin dashboard; team's established stack
Telephony	Plivo (primary); Exotel and Twilio adapters	India-focused pricing and DID availability; all providers sit behind a common abstraction layer with automatic failover
Speech-to-Text	Deepgram (streaming); Azure Speech fallback	Best-in-class latency and accuracy for telephone-grade audio; Indian English models available
Text-to-Speech	ElevenLabs Flash or Azure Neural (streaming)	Sub-300 ms first-byte synthesis, natural voice quality

Language models	Claude Sonnet (conversation reasoning and response generation); Claude Haiku (intent classification, summarization, fact extraction)	Two-tier model strategy: the high-capability model is reserved for decision-making; the fast, low-cost model handles high-volume background tasks, reducing cost approximately 10–20× on those workloads
Embeddings	OpenAI text-embedding-3-small	Knowledge base semantic search
Infrastructure	Docker Compose on Azure Ubuntu VM; Azure Blob Storage; Azure Key Vault	Deployed in the same region as the telephony provider's India endpoints to minimize audio round-trip latency
Observability	Prometheus, Grafana, structured logging (structlog)	Per-stage latency tracking, agent quality metrics, cost monitoring

All external providers (telephony, STT, TTS, LLM, embeddings) are integrated through internal interface abstractions with configured fallbacks, so any provider can be replaced or failed over without changes to application logic.

4. System Architecture



The system consists of four deployable components:

Core Platform (modular monolith). Hosts the decision engine and all business services: conversation management, planning, rule evaluation, tool execution, customer memory, knowledge retrieval, scheduling, CRM, tickets, campaigns, and audit. Implemented as a single FastAPI application with strict internal module boundaries — chosen over microservices because these modules share transactions and context on every

conversational turn, and separating them would introduce distributed-system complexity without benefit at target scale.

Voice Session Service (separate service). Manages live calls: telephony media streams, streaming speech recognition and synthesis, interruption and silence handling. Deployed separately because its scaling characteristics (I/O-bound audio streaming, WebSocket-heavy) differ from the core platform, and a fault in call handling must not affect the core engine.

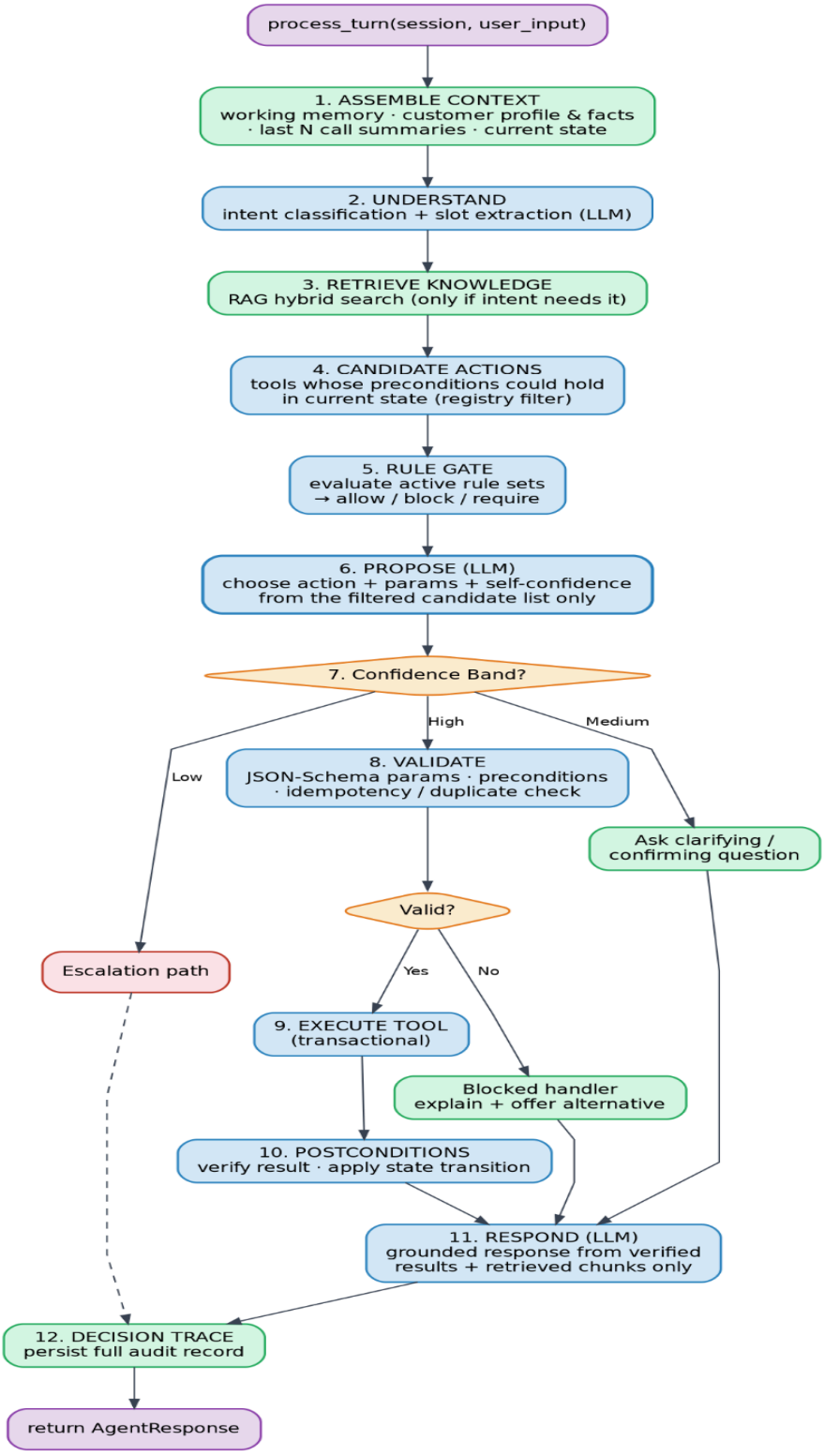
Worker pool. Celery workers for all asynchronous work: post-call summarization and fact extraction, campaign dialing, reminder and retry scheduling, knowledge document ingestion.

Admin Dashboard. React application for operations staff: customer and call management, live call monitoring, rule and knowledge base editing, campaign control, analytics, and audit trail review.

The two services communicate through a single frozen contract — `process_turn(conversation, user_input) → agent_response` — which allows voice-layer and engine development to proceed in parallel and enables additional channels (chat, WhatsApp) to reuse the same engine unchanged.

---

5. Conversation Processing: The Decision Cycle



Each customer utterance is processed through a fixed twelve-step cycle. Steps are implemented either as deterministic code or as LLM calls; the distinction is deliberate and central to the design. A typical turn requires only two to three LLM calls, which keeps both latency and per-call cost controlled.

Context assembly *(deterministic)*. The engine loads the customer profile, extracted facts, summaries of previous conversations, the current conversation history, and the customer's journey stage.

Intent understanding *(LLM — Claude Haiku)*. A fast classification call determines the customer's intent and extracts structured entities (dates, products, locations). Speech-recognition confidence from the voice layer is factored in at this stage.

Knowledge retrieval *(deterministic)*. For factual queries, the engine performs hybrid search over the knowledge base: pgvector semantic similarity combined with PostgreSQL full-text search, merged by reciprocal rank fusion. Documents outside their configured validity period (e.g., expired promotions) are excluded at query level.

Candidate action filtering *(deterministic)*. The tool registry is filtered to actions that are valid in the customer's current journey stage, enabled for the tenant, and permitted for the agent's role. The LLM is only ever presented with this filtered list — actions invalid in the current context are not visible to the model, which structurally eliminates entire categories of erroneous behavior.

Rule evaluation *(deterministic)*. All active business rules are evaluated. Rules may block actions, impose prerequisites (e.g., identity verification before account changes), or force escalation. Blocked actions are removed from the candidate list. Rule evaluation contains no LLM involvement and is fully repeatable.

Action proposal *(LLM — Claude Sonnet)*. The primary reasoning call. The model receives the conversation context, customer memory, retrieved knowledge, the filtered action list with parameter schemas, and the stage-specific objective. It returns a structured JSON proposal: chosen action, parameters, response draft, and a self-assessed confidence score.

Confidence gating *(deterministic)*. The model's self-assessment is adjusted by objective factors — speech-recognition quality, retrieval grounding strength, ambiguity between candidate actions, and repeated clarification attempts — producing a final score mapped to three bands: **high** (proceed to validation), **medium** (do not act; ask a confirming question), **low** (escalate to a human). Thresholds are tenant-configurable.

Validation *(deterministic)*. Three mandatory gates before any execution: parameter validation against the tool's JSON Schema; precondition checks against live data (slot existence and capacity, appointment ownership); and idempotency verification to prevent duplicate effects such as double-booking.

Execution *(deterministic)*. The action runs within a database transaction where applicable. Slot booking uses row-level locking to guarantee that concurrent calls cannot book the same capacity. Each tool defines a timeout and an explicit failure policy (retry, offer alternative, or escalate).

Postcondition verification and state transition *(deterministic)*. The engine verifies the intended effect exists in the database before reporting success. Only then does the customer's journey stage advance. A failed verification is treated as a failed action — success is never reported to the customer without database-level proof.

Response generation *(LLM — Claude Sonnet)*. The reply is generated from a grounding context containing only verified execution results and retrieved knowledge passages, with instructions prohibiting any figure, date, price, or time not present in that material. Combined with retrieval-level validity filtering, this constitutes the system's primary defense against fabricated information.

Decision trace *(deterministic)*. A complete audit record is persisted asynchronously: input, memory and knowledge references consumed, every rule evaluated with its verdict, the model's proposal and reasoning, validation results, confidence, action taken, prompt versions, model identifiers, token usage, and per-stage latency.

---

6. Core Engine Components

6.1 Tool Registry

Every executable action is defined by a declarative contract stored in the database, specifying: input schema (JSON Schema), journey stages in which the action is permitted, preconditions verified against live data, postconditions proving successful execution, effects (state transitions, emitted events), failure policy, timeout, and an idempotency key. Actions with irreversible effects (bookings, cancellations, ticket creation) are additionally flagged `requires_confirmation`, which forces an explicit customer confirmation turn before execution.

Contracts are configuration, not code: enabling a tool for a new tenant, adjusting its permitted stages, or integrating an external system (a client's own CRM exposed as an HTTP tool) requires no engine changes, and every tool — internal or external — passes through identical validation and audit machinery.

6.2 Rule Engine

Business policies are authored in YAML, versioned, validated at activation, and evaluated by deterministic code at three checkpoints: before any outbound dial, before any proposed action, and on every turn. Rules emit verdicts — block, require, escalate — and a block cannot be overridden by any other rule or by model reasoning. Standard rule sets ship with the platform (calling policy, retry policy, escalation policy, appointment policy, eligibility); tenants extend or modify them through the dashboard, with full version history and single-action rollback. Every rule evaluation, including non-matches, is written to the decision trace, making questions such as "why was this customer not called" answerable from a single record.

6.3 Customer Journey State Machine

Each customer occupies exactly one journey stage (Lead, Contacted, Qualified, Interested, Appointment Booked, Converted, and others), defined in configuration per industry pack. Each stage carries an objective that is injected into the reasoning prompt, which is how a single engine behaves as a sales agent for a lead and as a support agent for an open ticket. Stage transitions occur only as verified effects of executed tools or explicit signals (e.g., an opt-out request) — the model cannot assert a stage change. Interrupting stages (Call Later, Support Active) suspend and later restore the customer's position in the journey. Every transition is recorded with its trigger, actor, and confidence.

6.4 Memory Architecture

Memory type	Content	Storage	Written by
Working	Current conversation turns and extracted entities	Redis (session-scoped)	Live, per turn
Episodic	Structured summaries of past conversations: outcome, sentiment, next step	PostgreSQL	Post-call worker (Claude Haiku)
Semantic	Extracted customer facts: preferences, objections, pain points, buying intent — each with confidence and source-conversation provenance	PostgreSQL	Post-call extraction (Claude Haiku)
Organizational	Company knowledge: brochures, pricing, FAQs, policies, playbooks	PostgreSQL + pgvector	Ingestion pipeline

Facts are append-only with supersede links — a corrected fact retains its history — and supervisors can review or amend the agent's memory through the dashboard. The identifiers of every memory item consulted in a decision are recorded in that decision's trace.

6.5 Confidence and Escalation

Escalation to human agents is triggered either by the low-confidence band or by rule (VIP customers with negative sentiment, legal or payment-dispute indicators, repeated failed interactions, explicit customer request). The escalation form — immediate warm transfer, scheduled human callback, or ticket — is determined by the rule engine. For warm transfers, the system generates a context briefing (customer identity, journey stage, conversation summary, pending items) delivered to the receiving agent's console, and automated outbound contact with that customer is suspended until the escalation is resolved by a human.

---

7. Voice Processing Approach

The voice layer streams call audio to Deepgram for continuous transcription and streams generated replies through TTS, with end-of-turn detection combining transcription finality, voice-activity detection, and a configurable silence threshold. The end-to-end latency target is approximately **1.2 seconds** from the customer's last word to the first audio of the reply, achieved through: full streaming at every stage, speculative early processing of stable transcripts, sentence-level incremental synthesis, pre-generated audio for fixed phrases (disclosure, greetings, acknowledgments), and co-located deployment with the telephony provider's regional endpoints. Interruption handling (barge-in) halts agent audio within 150 ms of detected customer speech and informs the engine which portion of the reply was actually delivered. Full detail: *Voice Pipeline Design*.

---

8. Platform Genericity and Multi-Tenancy

The engine is domain-agnostic. All business-specific behavior — journey stages, rules, enabled tools, conversational persona, knowledge content — is delivered as an **industry pack**: a versioned configuration bundle imported per tenant. Onboarding a new client or vertical is a configuration exercise, not a development project. The data model is multi-tenant throughout (tenant-scoped rows, repository-level filtering, PostgreSQL row-level security as defense in depth), and per-tenant provider selection is supported (e.g., different telephony vendors per client).

---

9. Safety, Compliance, and Quality Assurance

Compliance controls are structural where possible: Do-Not-Contact is evaluated as the first, terminal rule before every dial with no bypassing code path; recording disclosure is enforced by the call session's state machine; consent is tracked in an append-only ledger with evidence references; PII is masked at the logging layer so raw identifiers cannot reach log files; and the model is never granted tools for destructive or privileged operations. Regulatory alignment covers TRAI/TCCCPR telemarketing requirements and DPDP Act data-protection principles. Full detail: *Compliance, Security & Audit Design*.

Quality assurance for agent behavior goes beyond conventional testing: a scripted conversation test suite asserts correct engine behavior across happy paths, failure branches, rule triggers, and adversarial inputs; a curated golden set of ~100 human-approved conversation turns is re-evaluated on every prompt or model change with human review of any behavioral diff; and production monitoring tracks escalation rate, blocked-action rate, grounding failures, and per-stage latency. Business KPIs (connection rate, conversion rate, appointments booked and completed, resolution times) and agent KPIs (action accuracy, latency, escalation rate) are computed from the same event and trace data. Full detail: *Deployment & Testing Strategy*.

---

10. Document Map

Document	Contents
Solution Overview & Engineering Approach (this document)	Consolidated summary for team and management
Architecture & Data Design	Full architecture, technology decisions, directory structure, complete database schema
Workflow Chart	End-to-end visual process flow
API Specification	Complete endpoint reference
Voice Pipeline Design	Telephony integration, streaming STT/TTS, latency engineering, call edge cases
Compliance, Security & Audit Design	Consent, DNC, PII protection, access control, audit trail
Deployment & Testing Strategy	Environments, CI/CD, monitoring, testing methodology, go-live checklist

Architecture & Data Design

Architecture & Data Design

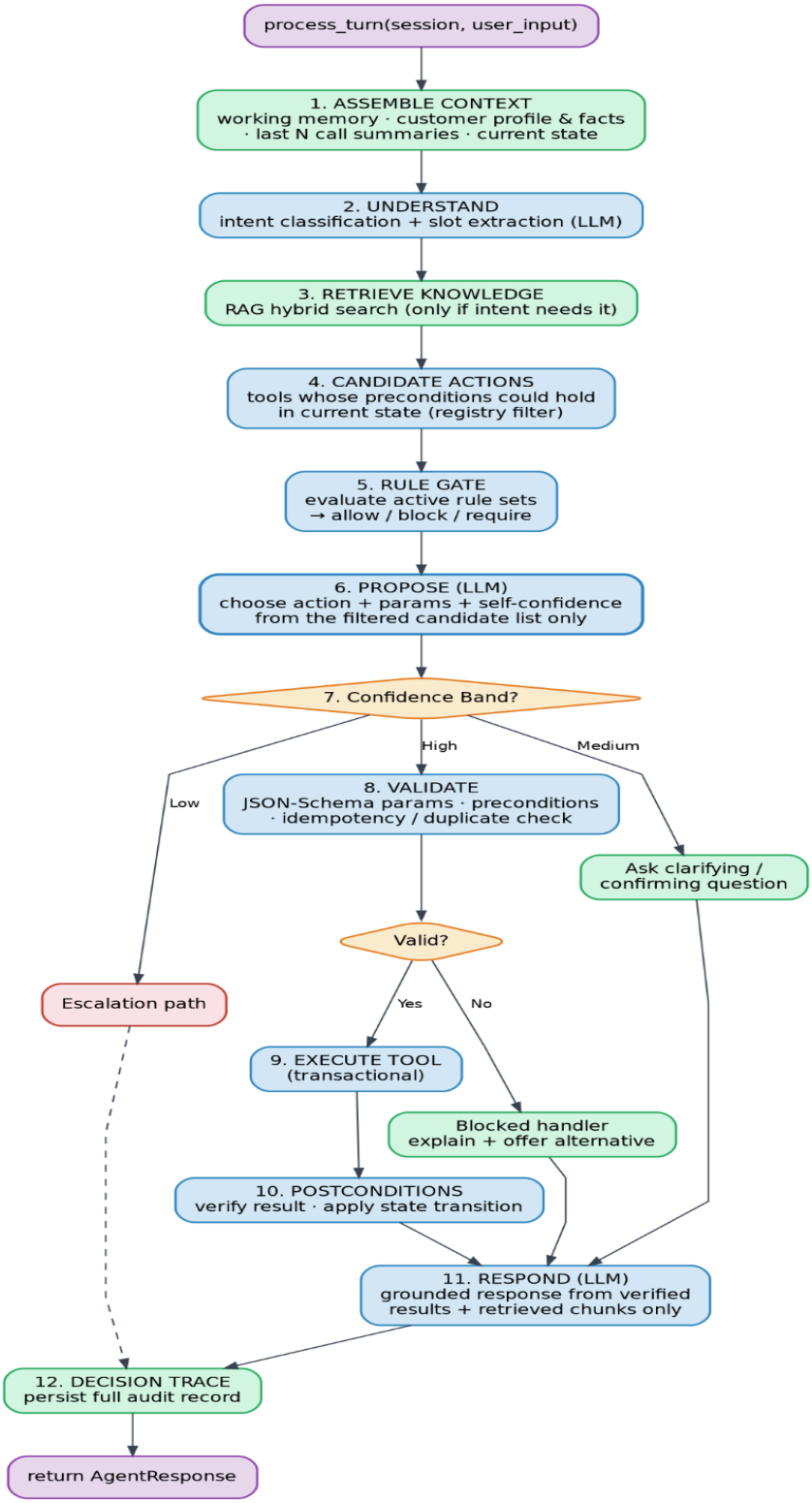
---

1. Design Principles

1. **Domain-agnostic core.** Nothing in the engine knows about "insurance" or "clinics". All domain behavior lives in **Industry Packs**: config bundles containing tools config, rules YAML, state machine definition, prompts, and knowledge documents. Deploying for a new vertical = new pack, zero engine code changes.
2. **The LLM never executes tools.** The LLM proposes; the Planner validates against the tool contract and Rule Engine, then the Executor runs it. (Detailed in the Solution Overview & Engineering Approach document.)
3. **Provider abstraction everywhere it hurts.** Telephony, STT, TTS, LLM, and embeddings are all behind interfaces with swappable adapters. Full abstraction layer (not thin adapter) — per full-scope decision.
4. **Everything auditable.** Every decision step emits an audit event with inputs, retrieved context, rules evaluated, and confidence. Explainability is a query over these events, not a reconstruction.
5. **Tenant-ready from day one.** Single-tenant deployments are just multi-tenant with one row in `tenants`. Every table carries `tenant_id`.
6. **Config over code.** States, rules, tools, prompts, calling policies — all data, editable via admin API without redeploy.

---

2. High-Level Architecture



2.1 Service Boundaries (what runs as what)

Unit	Deployment	Why separate / why not
Core Platform	1 FastAPI app (modular monolith), N replicas	All engine modules share transactions, memory, and the tool registry. Splitting them buys nothing at this scale and costs distributed-transaction pain.
Voice Session Service	Separate FastAPI/asyncio service	Different scaling profile (1 process ≈ N concurrent media streams, CPU-bound audio, WebSocket-heavy). Must be deployable close to telephony region. Crashes must not take down the core.
Worker Pool	Celery (or ARQ) workers	Post-call summarization, campaign dialing, reminder scheduling, document ingestion, retry timers.
Admin Dashboard	React 18 + Vite + Tailwind SPA	Your standard stack.
Postgres + pgvector	Managed (Azure Flexible Server) or VM	One database for relational + vector. No MongoDB — CRM/state/audit data is relational; conversation payloads fit JSONB.
Redis	Single instance + persistence	Session state (working memory), queues, rate limits, distributed locks (slot booking).
Object storage	Azure Blob / S3-compatible	Call recordings, raw transcripts, uploaded knowledge documents.

2.2 Provider Abstraction Layer (full scope)

Each external capability is a Python protocol with adapters registered in config:

```
providers:
  telephony:  { active: plivo,   fallback: exotel }
  stt:        { active: deepgram, fallback: azure_stt }
  tts:        { active: elevenlabs, fallback: azure_tts }
  llm:        { planner: claude-sonnet, summarizer: claude-haiku, fallback: gpt-4o }
  embeddings: { active: openai-3-small }
```

Interfaces (usage detail in the Voice Pipeline Design document):

- `TelephonyProvider`: `originate_call`, `answer`, `hangup`, `transfer`, `start_recording`, `send_dtmf`, webhook normalization → canonical `CallEvent`

- `STTProvider`: `stream_transcribe(audio_stream) → async iterator[TranscriptEvent]` (partial + final)
- `TTSProvider`: `stream_synthesize(text_stream) → async iterator[AudioChunk]`
- `LLMProvider`: `complete`, `stream_complete`, tool-call-shaped outputs normalized to internal schema
- `EmbeddingProvider`: `embed(texts) → vectors`

Canonical event/DTO models mean the engine never sees provider-specific payloads. Fallback routing: if the active provider errors ≥ threshold within a window, the circuit breaker flips to fallback and raises an ops alert.

---

3. Technology Stack

Layer	Choice	Notes
Core API	**Python 3.12 + FastAPI**	Your stack; async-first throughout
Voice service	FastAPI + `asyncio` + `websockets`	Media streaming over WS
Workers	**Celery + Redis** (or ARQ if you prefer pure-async)	Scheduled + queued jobs
DB	**PostgreSQL 16 + pgvector**	Relational + embeddings in one store
ORM / migrations	SQLAlchemy 2.0 (async) + Alembic	
Cache/queues/locks	**Redis 7**	
Object storage	Azure Blob	Recordings, docs
Telephony	**Plivo** (primary), Exotel adapter (India DID + TRAI compliance), Twilio adapter (global)	Behind abstraction layer
STT	**Deepgram** streaming (primary), Azure STT fallback	Best latency/accuracy for telephone audio; Indian-English models
TTS	**ElevenLabs Flash / Azure Neural**	Streaming, <300ms first-byte

LLM	**Claude Sonnet** (planner/conversation), **Claude Haiku** (summarization, extraction, classification)	Cheap model for high-volume background jobs
Embeddings	OpenAI text-embedding-3-small or voyage-3	1536-dim default
Frontend	React 18 + Vite + Tailwind	Your stack
Realtime dashboard	WebSocket (FastAPI)	Live call monitor
Deploy	Docker Compose on Azure Ubuntu VM (PM2 optional for node-less setup → prefer systemd/compose)	Kubernetes not needed at this scale
Observability	structlog + Prometheus + Grafana; audit events in Postgres	

---

4. Directory Structure

Monorepo, three deployables + shared package:

```
voice-agent-platform/
├── docker-compose.yml
├── .env.example
├── packages/
│   └── shared/                    # pip-installable internal package
│       ├── schemas/               # Pydantic DTOs shared across services
│       │   ├── call_events.py
│       │   ├── turns.py           # ProcessTurnRequest/Response — THE contract
│       │   ├── tools.py
│       │   └── memory.py
│       └── clients/
│           └── core_client.py     # typed client for core API (used by voice svc)
│
├── core/                          # CORE PLATFORM (modular monolith)
│   ├── app/
│   │   ├── main.py
│   │   ├── config.py              # pydantic-settings, env-driven
│   │   ├── api/                   # routers (see API Specification)
│   │   │   ├── v1/
│   │   │   │   ├── customers.py
│   │   │   │   ├── conversations.py   # process_turn lives here
│   │   │   │   ├── appointments.py
│   │   │   │   ├── scheduling.py
```

```
│   │   │   │   ├── knowledge.py
│   │   │   │   ├── rules.py
│   │   │   │   ├── tools_admin.py
│   │   │   │   ├── campaigns.py
│   │   │   │   ├── tickets.py
│   │   │   │   ├── calls.py
│   │   │   │   ├── analytics.py
│   │   │   │   ├── audit.py
│   │   │   │   └── auth.py
│   │   │   └── webhooks/
│   │   │       └── telephony.py      # normalized by provider adapter
│   │   ├── engine/                   # ★ the brain — domain-agnostic
│   │   │   ├── conversation_manager.py
│   │   │   ├── planner/
│   │   │   │   ├── planner.py        # intent → context → propose → validate → execute
│   │   │   │   ├── prompts/          # jinja2 prompt templates (pack-overridable)
│   │   │   │   └── confidence.py
│   │   │   ├── tools/
│   │   │   │   ├── registry.py       # loads tool contracts from DB/config
│   │   │   │   ├── executor.py
│   │   │   │   ├── contracts.py      # pre/postcondition validation
│   │   │   │   └── builtin/          # generic tool implementations
│   │   │   │       ├── crm_tools.py
│   │   │   │       ├── scheduling_tools.py
│   │   │   │       ├── knowledge_tools.py
│   │   │   │       ├── ticket_tools.py
│   │   │   │       └── escalation_tools.py
│   │   │   ├── rules/
│   │   │   │   ├── engine.py         # YAML rule evaluator
│   │   │   │   ├── conditions.py     # condition operators library
│   │   │   │   └── loader.py
│   │   │   ├── state_machine/
│   │   │   │   ├── machine.py        # config-driven FSM
│   │   │   │   └── transitions.py
│   │   │   ├── memory/
│   │   │   │   ├── service.py        # unified read API for planner
│   │   │   │   ├── working.py        # Redis-backed session memory
│   │   │   │   ├── episodic.py       # call summaries
│   │   │   │   ├── semantic.py       # extracted facts/preferences
│   │   │   │   └── pipeline.py       # Observe→Extract→Summarize→Update→Audit
│   │   │   └── rag/
│   │   │       ├── ingestion.py      # chunk, embed, store
│   │   │       ├── retrieval.py      # hybrid search + rerank
│   │   │       └── chunkers.py
│   │   ├── services/                 # non-engine business services
│   │   │   ├── crm.py
│   │   │   ├── scheduling.py         # slot computation, capacity, buffers
│   │   │   ├── campaigns.py
```

```
│   │   │   ├── tickets.py
│   │   │   ├── consent.py          # DND, consent ledger
│   │   │   ├── audit.py
│   │   │   └── analytics.py
│   │   ├── providers/              # ★ abstraction layer
│   │   │   ├── base.py             # protocols
│   │   │   ├── llm/       (anthropic.py, openai.py, router.py)
│   │   │   ├── telephony/ (plivo.py, exotel.py, twilio.py)
│   │   │   ├── stt/       (deepgram.py, azure.py)
│   │   │   ├── tts/       (elevenlabs.py, azure.py)
│   │   │   └── embeddings/(openai.py, voyage.py)
│   │   ├── db/
│   │   │   ├── models/             # SQLAlchemy models (mirror §5)
│   │   │   ├── repositories/
│   │   │   └── migrations/         # alembic
│   │   └── security/
│   │       ├── rbac.py
│   │       ├── pii.py              # masking pipeline
│   │       └── auth.py             # JWT, API keys
│   └── tests/
│
├── voice/                          # VOICE SESSION SERVICE
│   ├── app/
│   │   ├── main.py
│   │   ├── session/
│   │   │   ├── manager.py          # active call sessions
│   │   │   ├── pipeline.py         # audio in → STT → turn → core → TTS → audio out
│   │   │   ├── vad.py              # voice activity detection, silence policy
│   │   │   ├── barge_in.py
│   │   │   └── state.py            # per-call FSM (ringing/talking/listening/transferring)
│   │   ├── media/
│   │   │   ├── ws_handler.py       # provider media websockets
│   │   │   └── codecs.py           # μ-law/PCM handling
│   │   └── recording.py
│   └── tests/
│
├── workers/                        # background jobs
│   ├── tasks/
│   │   ├── post_call.py            # transcript → memory pipeline → CRM update
│   │   ├── campaigns.py            # outbound dial scheduling
│   │   ├── reminders.py            # appointment reminder calls
│   │   ├── retries.py              # busy/no-answer retry per rules
│   │   └── ingestion.py            # knowledge doc processing
│   └── beat_schedule.py
│
├── dashboard/                      # React admin
│   └── src/
│       ├── pages/  (Customers, Calls, LiveMonitor, Appointments, Knowledge,
```

```
│       │           Rules, Tools, Campaigns, Tickets, Analytics, Audit, Settings)
│       ├── components/
│       └── api/
│
└── packs/                      # ★ INDUSTRY PACKS (config, not code)
    ├── _template/
    │   ├── pack.yaml             # manifest
    │   ├── states.yaml           # state machine definition
    │   ├── rules/                # business rules
    │   ├── tools.yaml            # enabled tools + param overrides
    │   ├── prompts/              # persona, pitch style, objection handling
    │   └── knowledge/            # seed documents
    └── example_clinic/
```

---

5. Database Design (PostgreSQL 16 + pgvector)

Conventions: every table has `id UUID PK DEFAULT gen_random_uuid()`, `tenant_id UUID NOT NULL REFERENCES tenants(id)`, `created_at/updated_at TIMESTAMPTZ`. Indexes noted inline. All FKs implicit by name.

5.1 Platform & Identity

```
tenants (
  id, name, slug UNIQUE, industry_pack TEXT,        -- which pack is active
  timezone TEXT DEFAULT 'Asia/Kolkata',
  settings JSONB DEFAULT '{}',                      -- provider overrides, locale
  status TEXT CHECK (status IN ('active','suspended'))
)

users (                                             -- dashboard/human users
  id, tenant_id, email UNIQUE(tenant_id,email), password_hash,
  full_name, role TEXT,                             -- FK-ish to roles.name
  is_active BOOL, last_login_at
)

roles (
  id, tenant_id, name,                              -- admin, supervisor, agent, analyst
  permissions JSONB                                 -- ["customers:read","rules:write",...]
)

api_keys (
  id, tenant_id, name, key_hash, scopes JSONB, expires_at, last_used_at
)
```

5.2 CRM & Customer Memory

```
customers (
  id, tenant_id,
  full_name, phone TEXT NOT NULL, phone_normalized TEXT,   -- E.164
  email, location TEXT, region_code TEXT,
  preferred_language TEXT DEFAULT 'en',
  preferred_contact_start TIME, preferred_contact_end TIME,
  current_state TEXT NOT NULL DEFAULT 'lead',              -- FSM state (validated vs pack)
  lead_source TEXT, priority_score NUMERIC DEFAULT 0,
  assigned_executive_id UUID NULL,
  tags TEXT[], external_ref TEXT,                          -- link to client's own CRM
  UNIQUE (tenant_id, phone_normalized)
)
-- INDEX: (tenant_id, current_state), (tenant_id, priority_score DESC), GIN(tags)

customer_facts (                    -- SEMANTIC MEMORY: extracted, structured
  id, tenant_id, customer_id,
  category TEXT,                    -- pain_point | objection | preference |
                                    -- buying_intent | decision_maker | product_interest
  key TEXT, value JSONB,
  confidence NUMERIC,               -- extraction confidence 0..1
  source_conversation_id UUID NULL,    -- provenance
  status TEXT DEFAULT 'active',        -- active | superseded | retracted
  superseded_by UUID NULL
)
-- INDEX: (tenant_id, customer_id, category, status)

customer_product_interest (
  id, tenant_id, customer_id, product_ref TEXT,
  interest_level TEXT,              -- interested | purchased | rejected
  noted_at, source_conversation_id
)

state_transitions (
  id, tenant_id, customer_id,
  from_state TEXT, to_state TEXT,
  trigger TEXT,                     -- tool name / rule / manual / system
  tool_execution_id UUID NULL,
  actor TEXT,                       -- 'agent' | 'human:<user_id>' | 'system'
  confidence NUMERIC, reason TEXT, occurred_at TIMESTAMPTZ
)
-- INDEX: (tenant_id, customer_id, occurred_at DESC)
```

5.3 Conversations, Calls & Episodic Memory

```
conversations (                     -- channel-agnostic container
  id, tenant_id, customer_id,
```

```
  channel TEXT,                    -- voice | chat | whatsapp(future) | email(future)
  direction TEXT,                  -- inbound | outbound
  status TEXT,                     -- active | completed | abandoned
  started_at, ended_at,
  summary TEXT,                    -- EPISODIC MEMORY: post-call summary
  outcome TEXT,                    -- booked | callback_requested | not_interested | ...
  sentiment TEXT, intent_labels TEXT[],
  campaign_id UUID NULL
)
-- INDEX: (tenant_id, customer_id, started_at DESC)

messages (
  id, tenant_id, conversation_id,
  role TEXT,                       -- customer | agent | system | tool
  content TEXT,
  audio_offset_ms INT NULL,        -- position in recording
  tool_execution_id UUID NULL,
  metadata JSONB, created_at
)
-- INDEX: (conversation_id, created_at)

calls (                            -- telephony-specific facts for a conversation
  id, tenant_id, conversation_id, customer_id,
  provider TEXT, provider_call_id TEXT,
  from_number, to_number,
  status TEXT,                     -- queued|ringing|answered|busy|no_answer|failed|completed
  answered_at, ended_at, duration_seconds INT,
  hangup_cause TEXT, recording_url TEXT, transcript_url TEXT,
  disclosure_played BOOL DEFAULT false,
  retry_of_call_id UUID NULL, retry_reason TEXT,
  cost NUMERIC NULL
)
-- INDEX: (tenant_id, customer_id, created_at DESC), (provider, provider_call_id)

callback_requests (                -- "call me later"
  id, tenant_id, customer_id, requested_for TIMESTAMPTZ,
  source_conversation_id, status TEXT, -- pending | completed | cancelled
  fulfilled_by_call_id UUID NULL
)
```

5.4 Scheduling Engine

```
centres (
  id, tenant_id, name, address, city, region_code,
  lat NUMERIC, lng NUMERIC, phone, timezone,
  is_active BOOL
)
```

```
executives (                        -- bookable humans
  id, tenant_id, centre_id, full_name, email, phone,
  specialties TEXT[], is_active BOOL
)

working_hours (
  id, tenant_id,
  owner_type TEXT, owner_id UUID,      -- centre | executive
  day_of_week INT,                     -- 0-6
  start_time TIME, end_time TIME
)

holidays (
  id, tenant_id, owner_type, owner_id NULL,  -- NULL = tenant-wide
  date DATE, name TEXT
)

slot_policies (
  id, tenant_id, centre_id NULL,
  slot_duration_min INT, buffer_before_min INT, buffer_after_min INT,
  max_parallel INT,                    -- capacity per slot
  booking_horizon_days INT, min_notice_hours INT
)

appointments (
  id, tenant_id, customer_id, centre_id, executive_id NULL,
  starts_at TIMESTAMPTZ, ends_at TIMESTAMPTZ,
  status TEXT,                         -- booked|rescheduled|cancelled|completed|no_show
  booked_via TEXT,                     -- agent | dashboard | api
  source_conversation_id UUID NULL,
  reschedule_of UUID NULL, cancel_reason TEXT,
  reminder_state JSONB DEFAULT '{}'    -- {t24h: sent, t2h: pending}
)
-- INDEX: (tenant_id, centre_id, starts_at), (tenant_id, customer_id),
--        (tenant_id, executive_id, starts_at)
-- Slot availability = computed (working_hours − holidays − appointments
--   respecting slot_policies), never materialized. Booking uses
--   SELECT ... FOR UPDATE + capacity check in one transaction (+ Redis lock).
```

5.5 Knowledge (RAG)

```
knowledge_documents (
  id, tenant_id, title, doc_type TEXT,   --
brochure|faq|pricing|policy|promo|playbook|sop|compliance
  source_uri TEXT,                     -- blob path
  status TEXT,                         -- processing | ready | failed
```

```
  version INT, effective_from DATE NULL, effective_to DATE NULL,  -- promos expire
  metadata JSONB
)

knowledge_chunks (
  id, tenant_id, document_id,
  chunk_index INT, content TEXT,
  embedding VECTOR(1536),
  token_count INT, metadata JSONB        -- headings, page, product refs
)
-- INDEX: HNSW (embedding vector_cosine_ops), (tenant_id, document_id)
-- Hybrid search: pgvector cosine + tsvector GIN on content, RRF-merged (see Solution
Overview, Section 5)
```

5.6 Engine Configuration (tools, rules, states, prompts)

```
tool_definitions (                    -- the TOOL REGISTRY (contract: Solution Overview 6.1)
  id, tenant_id, name UNIQUE(tenant_id,name), description,
  category TEXT,                      -- sales|scheduling|support|utility
  input_schema JSONB,                   -- JSON Schema: required/optional params
  preconditions JSONB,                -- declarative checks (Solution Overview 6.1)
  postconditions JSONB,
  failure_policy JSONB,               -- retry, fallback tool, escalate
  requires_confirmation BOOL,           -- read back to customer before executing
  permission_level TEXT,              -- which roles / agent may invoke
  implementation TEXT,                -- python path of builtin, or 'http' for external
  http_config JSONB NULL,               -- for external tool endpoints
  is_enabled BOOL
)

rule_sets (
  id, tenant_id, name, category TEXT,    --
calling_policy|retry|dnd|escalation|appointment|eligibility
  yaml_source TEXT,                    -- authored YAML (see Solution Overview 6.2)
  compiled JSONB,                     -- parsed/validated form the engine evaluates
  version INT, is_active BOOL, activated_by UUID, activated_at
)
-- history kept: new version = new row; is_active flips

state_machine_defs (
  id, tenant_id, version INT, is_active BOOL,
  definition JSONB                    -- states, allowed transitions, entry actions
)

prompt_templates (
  id, tenant_id, name, purpose TEXT,     -- persona|planner|summarizer|objection|pitch
  template TEXT, model_hint TEXT, version INT, is_active BOOL
```

)

5.7 Support, Campaigns, Consent

```
tickets (
  id, tenant_id, customer_id, conversation_id NULL,
  subject, description, category TEXT, priority TEXT,
  status TEXT,                        -- open|in_progress|resolved|escalated|closed
  assigned_to UUID NULL, resolution TEXT, resolved_at,
  sla_due_at TIMESTAMPTZ NULL
)

escalations (
  id, tenant_id, customer_id, conversation_id, ticket_id NULL,
  reason TEXT,                         --
low_confidence|failed_interactions|legal|vip|payment_dispute|customer_request
  trigger_rule TEXT NULL, confidence NUMERIC NULL,
  status TEXT,                        -- pending|accepted|resolved
  assigned_to UUID NULL, transferred_call_id UUID NULL
)

campaigns (
  id, tenant_id, name, status,          -- draft|running|paused|completed
  audience_filter JSONB,                -- CRM query definition
  pack_prompt_override UUID NULL,
  schedule JSONB,                       -- windows, pacing (calls/hour), start/end dates
  stats JSONB                           -- denormalized counters
)

campaign_targets (
  id, tenant_id, campaign_id, customer_id,
  status TEXT,                        -- pending|calling|done|failed|excluded
  attempts INT, last_attempt_at, exclusion_reason TEXT
)

consent_records (                     -- append-only consent ledger
  id, tenant_id, customer_id,
  consent_type TEXT,                    -- calls|recording|marketing|data_processing
  granted BOOL, channel TEXT, evidence_uri TEXT,  -- recording clip / form ref
  recorded_at TIMESTAMPTZ
)

dnc_list (                            -- Do Not Contact
  id, tenant_id, phone_normalized, customer_id NULL,
  scope TEXT,                         -- all|marketing|voice
  source TEXT,                        -- customer_request|regulator|import
  added_at, expires_at NULL
```

```
)
-- INDEX: UNIQUE (tenant_id, phone_normalized, scope)
```

5.8 Audit & Analytics

```
decision_traces (                    -- EXPLAINABILITY core (one per planner cycle)
  id, tenant_id, conversation_id, customer_id, turn_index INT,
  input_text TEXT,
  retrieved_memory JSONB,            -- fact/summary ids + snippets used
  retrieved_chunks JSONB,            -- knowledge chunk ids + scores
  rules_evaluated JSONB,             -- [{rule, result, reason}]
  planner_reasoning TEXT,
  proposed_tool TEXT NULL, tool_params JSONB NULL,
  validation_result JSONB,           -- pre/postcondition outcomes
  confidence NUMERIC, confidence_band TEXT,   -- high|medium|low
  action_taken TEXT,                 -- executed|clarified|escalated|blocked
  state_transition_id UUID NULL,
  latency_ms INT, model_used TEXT, tokens JSONB,
  created_at
)
-- INDEX: (tenant_id, conversation_id, turn_index), (tenant_id, created_at)
-- Partition by month at scale.

tool_executions (
  id, tenant_id, decision_trace_id, tool_name,
  input JSONB, output JSONB, status TEXT,   -- success|failed|blocked
  error TEXT NULL, duration_ms INT, created_at
)

audit_events (                       -- generic security/admin audit
  id, tenant_id, actor TEXT, action TEXT, resource_type, resource_id,
  before JSONB NULL, after JSONB NULL, ip TEXT, created_at
)

analytics_events (                   -- append-only event stream for KPIs
  id, tenant_id, event_type TEXT,    -- call_placed|call_connected|appointment_booked|...
  customer_id NULL, conversation_id NULL, properties JSONB, occurred_at
)
-- KPIs (connection rate, conversion, tool accuracy, escalation rate, CSAT,
-- hallucination flags) are SQL aggregations / materialized views over this
-- + decision_traces. No separate analytics DB in v1.
```

---

6. Memory Architecture → Storage Mapping

PRD Memory	Storage	Written by	Read by planner as
Working (current conversation)	Redis hash per session (`session:{conv_id}`): rolling turn window, extracted slots, current intent. TTL = call + 1h. Flushed to `messages` continuously.	Conversation Manager, live	Full context block
Episodic (past conversations)	`conversations.summary` + `outcome`, `sentiment`	Post-call worker (Haiku summarization)	Last N summaries (default 5)
Semantic (profile, facts, preferences)	`customers` + `customer_facts` + `customer_product_interest`	Post-call extraction job; also live extraction for critical facts (e.g., "call me after 6pm")	Structured profile block
Organizational (shared knowledge)	`knowledge_chunks` (pgvector)	Ingestion worker	Top-k retrieved chunks

Full pipeline (Observe → Extract → Summarize → Update → Audit):

1. **Observe** — every turn appended to `messages`; working memory updated in Redis.
2. **Extract** (post-call, + inline for critical facts) — Haiku pass over transcript → candidate facts with confidence, category, provenance. Facts below confidence threshold are stored as `status='candidate'` for human review rather than discarded.
3. **Summarize** — structured summary (outcome, objections, sentiment, next-step) → `conversations.summary`.
4. **Update** — facts merged into `customer_facts` with supersede logic (new value ≠ old → old row `superseded`, linked via `superseded_by`; never destructive). Profile fields (preferred time, language) promoted when confidence ≥ threshold.
5. **Audit** — memory-write events into `audit_events`; every planner read logs the exact fact/summary IDs consumed into `decision_traces.retrieved_memory` — this is what makes "which memories influenced this decision" a single query.

---

7. Multi-Tenancy & Industry Packs

- **Isolation:** shared schema, `tenant_id` on every row, enforced by a repository-layer tenant filter (every query goes through repositories; raw session access is

lint-blocked). Postgres RLS enabled as defense-in-depth for `customers`, `customer_facts`, `consent_records`, `dnc_list`.

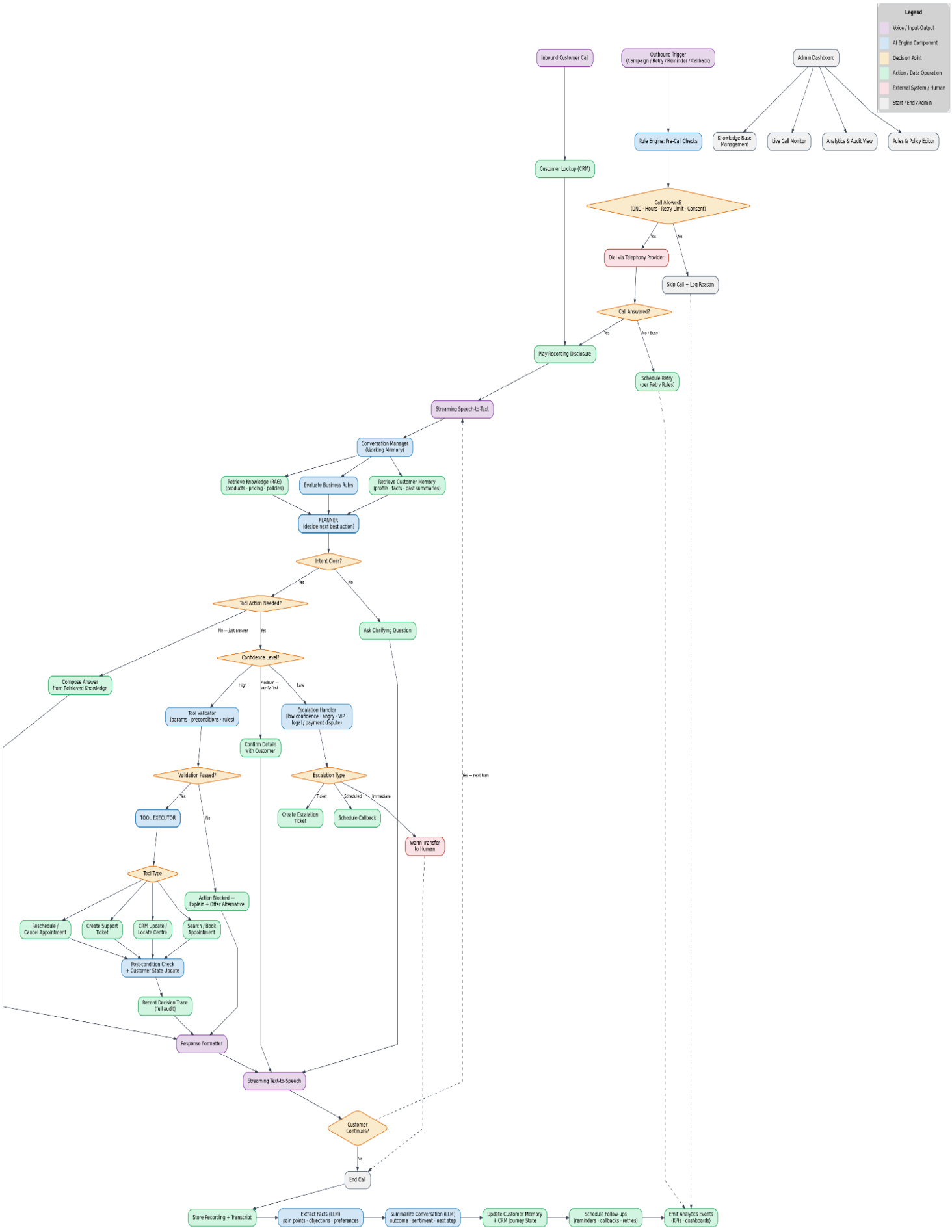
- **Pack loading:** `tenants.industry_pack` names a pack; on activation, pack YAMLs are imported into `state_machine_defs`, `rule_sets`, `tool_definitions`, `prompt_templates` as versioned rows. Runtime reads only from DB — packs are seeders, not runtime dependencies. Editing via dashboard just creates new versions.
- **Per-tenant provider overrides** in `tenants.settings` (e.g., one tenant on Exotel, another on Plivo; different TTS voice).
- **Single-tenant deployment** (your likely v1) = one tenant row. Nothing else changes.

---

8. Key Architectural Decisions (summary)

#	Decision	Rationale
AD-1	Modular monolith + separate voice service	Latency/scaling isolation for media; transactional simplicity for engine
AD-2	Postgres + pgvector only (no MongoDB)	Data is relational; JSONB covers flexible payloads; one DB to operate
AD-3	Full provider abstraction with fallback routing	Full-scope requirement; also de-risks India telephony vendor churn
AD-4	Slot availability computed, never materialized	Eliminates slot-table sync bugs; booking correctness via row locks
AD-5	All engine config (states/rules/tools/prompts) in versioned DB rows	Domain-agnostic platform; edit without redeploy; full audit history
AD-6	`decision_traces` as first-class table	Explainability is a query, not log spelunking
AD-7	Facts are append-only with supersede links	Memory is auditable and reversible; no silent overwrites
AD-8	Haiku-class model for all background LLM jobs	10–20x cost reduction on summarization/extraction volume

Workflow Chart


Legend
Voice / Input-Output
AI Engine Component
Decision Point
Action / Data Operation
External System / Human
Start / End / Admin
Inbound Customer Call
Outbound Trigger (Campaign / Retry / Reminder / Callback)
Admin Dashboard
Knowledge Base Management
Live Call Monitor
Analytics & Audit View
Rules & Policy Editor
Rule Engine: Pre-Call Checks
Customer Lookup (CRM)
Call Allowed? (DNC · Hours · Retry Limit · Consent)
Yes
No
Dial via Telephony Provider
Skip Call + Log Reason
Call Answered?
Yes
No / Busy
Play Recording Disclosure
Schedule Retry (per Retry Rules)
Streaming Speech-to-Text
Conversation Manager (Working Memory)
Retrieve Knowledge (RAG) (products · pricing · policies)
Evaluate Business Rules
Retrieve Customer Memory (profile · facts · past summaries)
PLANNER (decide next best action)
Intent Clear?
Yes
No
Tool Action Needed?
No — just answer
Yes
Ask Clarifying Question
Confidence Level?
High
Medium — verify first
Low
Compose Answer from Retrieved Knowledge
Tool Validator (params · preconditions · rules)
Escalation Handler (low confidence · angry · VIP · legal / payment dispute)
Confirm Details with Customer
Validation Passed?
Yes
No
Escalation Type
Yes — next turn
T-1 bot
Scheduled
Immediate
TOOL EXECUTOR
Create Escalation Ticket
Schedule Callback
Warm Transfer to Human
Tool Type
Action Blocked — Explain + Offer Alternative
Reschedule / Cancel Appointment
Create Support Ticket
CRM Update / Locate Centre
Search / Book Appointment
Post-condition Check + Customer State Update
Record Decision Trace (full audit)
Response Formatter
Streaming Text-to-Speech
Customer Continues?
No
End Call
Store Recording + Transcript
Extract Facts (LLM) pain points · objections · preferences
Summarize Conversation (LLM) outcome · sentiment · next step
Update Customer Memory + CRM Journey State
Schedule Follow-ups (reminders · callbacks · retries)
Emit Analytics Events (KPIs · dashboards)

API Specification

API Specification

---

1. The Basics (Read This First)

Base URL: `https://<your-domain>/api/v1`

Format: Everything is JSON in, JSON out. Timestamps are ISO-8601 with timezone (`2026-07-18T10:30:00+05:30`). IDs are UUIDs.

Who calls this API:

- The **React dashboard** (humans managing the system)
- The **Voice Session Service** (the phone-call handler — it mainly calls `process_turn`)
- The **telephony provider** (Plivo/Exotel sends us call events via webhooks)
- **External systems** of a client (their CRM syncing customers, for example)

Authentication — two kinds

Kind	Used by	How
JWT (login token)	Dashboard users	`POST /auth/login` with email + password → returns an access token (30 min) + refresh token. Send it as `Authorization: Bearer <token>`
API key	Services and integrations	Created in the dashboard with limited scopes (e.g., "can read customers, can't edit rules"). Sent as `X-API-Key: <key>`

Every request runs as one **tenant** (business). The tenant is inside the token/key — you never pass tenant_id manually, and you can never see another tenant's data.

Permissions: every user has a role (`admin`, `supervisor`, `agent`, `analyst`), and every endpoint declares what role it needs. Trying something above your role → `403`.

Errors — one shape everywhere

```
{
  "error": {
    "code": "SLOT_NOT_AVAILABLE",
    "message": "The requested slot is no longer available.",
    "details": { "requested_slot": "2026-07-18T10:30:00+05:30" },
    "request_id": "req_8f3a..."
```

```
  }
}
```

`request_id` appears in our logs too — quote it when reporting a problem. Standard HTTP codes: `400` bad input · `401` not logged in · `403` not allowed · `404` not found · `409` conflict (e.g., double booking) · `422` validation failed · `429` too many requests · `500` our bug.

Lists — one pattern everywhere

All list endpoints accept `?page=1&page_size=25&sort=-created_at` plus endpoint-specific filters, and return:

```
{ "items": [...], "page": 1, "page_size": 25, "total": 312 }
```

---

2. The Conversation API (the engine's front door)

`POST /conversations/{id}/turns` — process one turn

This is **the** endpoint — everything else is management around it. The voice service (or a chat client) calls it every time the customer says something.

Request:

```
{
  "user_input": "Saturday morning works for me",
  "channel": "voice",
  "input_metadata": { "stt_confidence": 0.91, "interrupted": false }
}
```

Response:

```
{
  "response_text": "Great! Just to confirm — Saturday July 18th, 10:30 AM at our Vapi centre. Shall I book it?",
  "response_segments": ["Great!", "Just to confirm — Saturday July 18th, 10:30 AM at our Vapi centre.", "Shall I book it?"],
  "action": "confirm",
  "executed_tools": [],
  "state_changed_to": null,
  "escalation": null,
  "end_call": false,
  "trace_id": "9d2c..."
}
```

`response_segments` exists so the voice layer can start speaking sentence 1 while the rest is still being processed — this is a latency trick, explained in the Voice Pipeline doc.

Other conversation endpoints

Method & path	What it does	Who uses it
`POST /conversations`	Start a conversation (body: customer_id or phone, channel, direction)	Voice service, chat
`POST /conversations/{id}/end`	Close it (body: reason). Triggers the post-call pipeline: summary, fact extraction, memory update	Voice service
`GET /conversations/{id}`	Full detail: messages, tools run, outcome, summary	Dashboard
`GET /conversations`	List/filter by customer, channel, outcome, date	Dashboard
`GET /conversations/{id}/messages`	Just the transcript	Dashboard

---

3. Customers (CRM)

Method & path	What it does
`POST /customers`	Create a customer (name, phone required; the rest optional)
`GET /customers/{id}`	Full profile including current journey stage
`GET /customers`	List with filters: `?state=interested&tag=vip&search=sharma&language=hi`
`PATCH /customers/{id}`	Edit profile fields
`DELETE /customers/{id}`	Soft-delete (kept for audit, excluded from everything)

`GET /customers/lookup?phone=+91...`	Find by phone — used on inbound calls to recognize the caller
`POST /customers/import`	Bulk import from CSV (returns a job id; see §12)

The customer's memory and journey

Method & path	What it does
`GET /customers/{id}/facts`	Everything the AI has learned: pain points, objections, preferences — each with confidence and which conversation it came from
`PATCH /customers/{id}/facts/{fact_id}`	Correct or retract a fact (humans can fix the AI's memory)
`GET /customers/{id}/timeline`	The full story in order: calls, stage changes, appointments, tickets
`GET /customers/{id}/state-history`	Just the journey stage changes, with what triggered each
`POST /customers/{id}/state`	Manually move a customer's stage (body: `to_state`, `reason`) — supervisor only, recorded as a manual transition

---

4. Scheduling & Appointments

Finding slots

`GET /scheduling/slots?centre_id=...&from=2026-07-18&to=2026-07-21&executive_id=(optional)`

Returns real, bookable slots — already accounting for working hours, holidays, buffer times, capacity, and existing bookings:

{ "slots": [
  { "starts_at": "2026-07-18T10:30:00+05:30", "ends_at": "...", "capacity_left": 2 },
  { "starts_at": "2026-07-18T11:30:00+05:30", "ends_at": "...", "capacity_left": 1 }

]}

This same query is what the AI's booking tool uses internally — dashboard and agent always see identical availability.

Appointments

Method & path	What it does
`POST /appointments`	Book (customer_id, centre_id, starts_at). Returns `409 SLOT_NOT_AVAILABLE` if taken in the meantime
`PATCH /appointments/{id}/reschedule`	Move to a new slot (old one is released atomically)
`POST /appointments/{id}/cancel`	Cancel with reason
`POST /appointments/{id}/complete`	Mark attended / no-show (feeds the journey stage)
`GET /appointments`	List/filter by centre, customer, date, status

Setup (admin)

CRUD endpoints, all following the same pattern (`POST`/`GET`/`PATCH`/`DELETE`):

`/centres` · `/executives` · `/working-hours` · `/holidays` · `/slot-policies` (slot length, buffers, capacity, how far ahead booking is allowed)

---

5. Knowledge Base (what the AI can quote)

Method & path	What it does
`POST /knowledge/documents`	Upload (multipart file or raw text) + metadata: type (`faq/pricing/policy/promo/...`), and optional `effective_from/to` dates. Processing is async — returns a job id
`GET /knowledge/documents`	List with processing status (`processing / ready / failed`)

`GET /knowledge/documents/{id}`	Detail + its chunks
`PATCH /knowledge/documents/{id}`	Update metadata / expiry dates
`DELETE /knowledge/documents/{id}`	Remove (its content immediately stops being retrievable)
`POST /knowledge/search`	Test retrieval by hand: `{ "query": "cancellation charges" }` → the exact chunks + scores the AI would see. Essential for debugging "why did the AI say that?"

The `effective_from/to` dates are how promotions safely expire: after the date passes, the content is automatically excluded from every search.

---

6. Engine Configuration

Rules

Method & path	What it does
`GET /rules`	All rule sets with active version numbers
`GET /rules/{name}`	The YAML of the active version
`PUT /rules/{name}`	Submit new YAML → validated → saved as a **new version** (not yet live)
`POST /rules/{name}/activate`	Make a version live (body: `version`). Rollback = activate the older version
`POST /rules/{name}/test`	Dry-run: send a sample situation, get back which rules would fire and the verdicts — test before going live
`GET /rules/{name}/versions`	Full history: who changed what, when

Tools

Method & path	What it does
`GET /tools`	The registry: every tool, enabled or not, its contract
`PATCH /tools/{name}`	Enable/disable, edit settings (e.g., turn `requires_confirmation` on)
`POST /tools`	Register an external HTTP tool (their endpoint + contract)
`POST /tools/{name}/test`	Dry-run the validation pipeline with sample params — see exactly which check would fail, without executing

States, prompts, packs

Method & path	What it does
`GET /state-machine` / `PUT /state-machine`	View / update the journey definition (versioned, validated on save)
`GET /prompts` / `PUT /prompts/{name}`	View / edit persona and task prompts (versioned)
`POST /packs/{name}/apply`	Load an industry pack into this tenant (imports its states, rules, tools, prompts as new versions)

---

7. Campaigns (outbound calling at scale)

Method & path	What it does
`POST /campaigns`	Create: name, audience filter (e.g., `state=lead, region=GJ`), calling window, pacing (calls/hour), start/end dates
`POST /campaigns/{id}/preview`	Shows who would be called — count + sample, **after** removing DNC and rule-blocked customers — before anything dials
`POST /campaigns/{id}/start` · `/pause` · `/stop`	Control
`GET /campaigns/{id}`	Live stats: queued, calling, connected, converted, failed

`GET /campaigns/{id}/targets`	Per-customer status inside the campaign, incl. exclusion reasons

Every single dial still passes the pre-call rule check individually at dial time — a campaign cannot bypass DNC or calling hours even if it was created before a customer opted out.

---

8. Support & Escalations

Method & path	What it does
`POST /tickets` · `GET /tickets` · `GET /tickets/{id}` · `PATCH /tickets/{id}`	Standard ticket CRUD: category, priority, status, assignment, resolution. (The AI creates tickets through its tool; these endpoints are for humans)
`GET /escalations`	The human work queue: everything the AI handed over, with reason and urgency
`POST /escalations/{id}/accept`	A human takes it (also delivers the AI-written context briefing)
`POST /escalations/{id}/resolve`	Close it — this also unlocks the customer for automated contact again

---

9. Calls, Callbacks & Telephony

Method & path	What it does
`GET /calls`	Call log: filter by customer, status, direction, date
`GET /calls/{id}`	Detail: duration, outcome, retry chain, cost
`GET /calls/{id}/recording`	Short-lived signed URL to the audio (access is logged)
`POST /calls/outbound`	Manually trigger one outbound call to a customer (passes pre-call rules like everything else)
`GET /callbacks` · `PATCH /callbacks/{id}`	The "call me later" queue: view, reschedule, cancel

Webhooks (telephony provider → us)

`POST /webhooks/telephony/{provider}` — Plivo/Exotel/Twilio send call events here (ringing, answered, hangup, recording ready). The provider adapter translates each provider's format into our canonical events. Secured by provider signature verification, not by API key.

---

10. Consent & Do-Not-Contact

Method & path	What it does
`GET /customers/{id}/consents`	The consent ledger: what was granted/revoked, when, with what evidence
`POST /customers/{id}/consents`	Record a consent event (type: calls / recording / marketing; granted: true/false)
`GET /dnc`	The Do-Not-Contact list
`POST /dnc`	Add a number (source: customer request / regulator / import) — takes effect immediately, everywhere
`DELETE /dnc/{id}`	Remove — admin only, and the removal itself is audit-logged
`POST /dnc/import`	Bulk import (e.g., national DNC registry file)

---

11. Analytics & Audit

Analytics

Method & path	What it does
`GET /analytics/overview?from=&to=`	The headline numbers: calls made, connection rate, conversion rate, appointments booked & completed, tickets resolved
`GET /analytics/funnel`	Journey funnel: how many customers sit in each stage, and conversion between stages
`GET /analytics/agent`	AI quality numbers: tool-choice accuracy, average confidence, escalation rate, average response latency, hallucination flags

`GET /analytics/campaigns/{id}`	Per-campaign results
`GET /analytics/export?from=&to=&format=csv`	Raw event export

Audit & explainability ("why did it do that?")

Method & path	What it does
`GET /audit/traces/{trace_id}`	The complete record of one decision: what the customer said, what memory and knowledge was used, every rule checked and its verdict, what the AI proposed and why, what validation said, confidence, what happened. This is the flight recorder readout
`GET /audit/traces?conversation_id=`	All decisions in one conversation, in order
`GET /audit/blocked?from=&to=`	Every action that was blocked, and by which rule — the compliance report
`GET /audit/events`	Admin activity log: who changed which rule/setting, when, old → new value

---

12. Everything Else

Auth & users: `POST /auth/login` · `POST /auth/refresh` · `POST /auth/logout` · CRUD on `/users` and `/roles` · CRUD on `/api-keys` (admin).

Async jobs: long tasks (imports, document processing) return `{ "job_id": ... }` immediately. Check progress: `GET /jobs/{id}` → `{ status: "processing" | "done" | "failed", progress: 0.7, result: ... }`.

Health: `GET /health` (is the API up) · `GET /health/deep` (DB, Redis, providers — for monitoring).

Live updates (WebSocket)

`WS /ws/dashboard` — the dashboard subscribes and receives pushed events, so screens update without refreshing:

Event	When
`call.started` / `call.ended`	Live call monitor
`call.turn`	A live call's latest exchange (for the "listen in" view)
`escalation.created`	A human is needed — this is the alert that matters
`appointment.booked`	Bookings ticker
`campaign.progress`	Campaign counters

---

13. One Worked Example (tying it together)

A typical outbound sales call touches the API like this:

1. Campaign worker: pre-call rules pass → tells telephony to dial
2. Provider → `POST /webhooks/telephony/plivo` (call answered)
3. Voice service → `POST /conversations` (start, direction: outbound)
4. Voice service → `POST /conversations/{id}/turns` — repeated for every exchange; one of these turns internally books the appointment via the booking tool
5. Voice service → `POST /conversations/{id}/end` — triggers summary + memory update in the background
6. Dashboard (already connected to `WS /ws/dashboard`) shows `appointment.booked` in real time
7. A supervisor later opens `GET /audit/traces?conversation_id=...` and reads exactly why the AI offered Saturday instead of Friday

Voice Pipeline Design

Voice Pipeline Design

---

1. What This Document Covers

Everything between the telephone network and the brain:

- How a phone call reaches our system (and how we place calls)
- How the customer's speech becomes text, and our text becomes speech
- The **one-second problem** — why voice is hard, and our plan to keep replies fast
- Handling real conversation behavior: interruptions, silences, background noise
- Recording, disclosures, transfers to humans, and what happens when things break

This is the domain of the **Voice Session Service** — the separate service from the architecture doc. One instance of it handles many simultaneous calls; each call lives inside a "session" object that owns everything about that call.

---

2. The Cast of Characters

Telephony provider (Plivo / Exotel / Twilio). The company that connects us to the actual phone network. They give us phone numbers, place and receive calls for us, and — critically — give us a **live audio stream** of the call over a WebSocket, in both directions. We hear the caller through it, and whatever audio we push into it, the caller hears.

STT — Speech-to-Text (Deepgram, Azure fallback). We stream the caller's audio to it continuously, and it streams text back *as the person speaks* — first as rough "partial" guesses, then a "final" version once the sentence is complete. Like live subtitles being typed and corrected in real time.

TTS — Text-to-Speech (ElevenLabs / Azure Neural). The reverse: we stream text in, it streams natural-sounding audio out. The important property we need: it starts producing audio within ~200 ms of receiving the first words, rather than waiting for the whole sentence.

The brain (Core Platform). Receives the finished text of each customer turn via `process_turn`, returns what to say. Covered fully in the Core Engine doc.

All three external pieces sit behind our provider abstraction layer — each can be swapped or failed-over without touching pipeline logic.

---

3. Life of a Call

Inbound (customer calls us)

1. Provider receives the call on our number → sends a webhook: "incoming call from +91-XXXXX."
2. We look up the caller: `GET /customers/lookup?phone=...`. Known customer → their file is preloaded. Unknown → a new lead record is created on the spot.
3. We answer and immediately open the media stream (the two-way audio WebSocket).
4. **The disclosure plays first** — "This call may be recorded..." — from a pre-generated audio file (no TTS delay), and the call record is marked `disclosure_played = true`. Recording starts only after this.
5. A conversation is opened in the core (`POST /conversations`), and the greeting is spoken. The turn loop (Section 4) begins.

Outbound (we call the customer)

1. A trigger fires: campaign, retry timer, reminder, or a "call me later" coming due.
2. The worker asks the core to run **pre-call rule checks** (DNC, calling hours, retry limits, consent). Any block → the call never happens, the reason is logged.
3. Checks pass → we tell the provider to dial.
4. The provider reports what happened: **answered** → same as inbound from step 3 onward. **Busy / no answer** → a retry is scheduled per the retry rules. **Voicemail detected** → hang up without speaking (configurable per tenant: some prefer leaving a short message).
5. One extra politeness step for outbound: the agent confirms it's speaking to the right person before anything else ("Am I speaking with Mr. Sharma?").

Ending

Either side hangs up, or the brain returns `end_call: true` (conversation concluded politely). We close the media stream, tell the core (`POST /conversations/{id}/end`), and the post-call pipeline (summary, fact extraction, memory update) runs in the background. The recording is uploaded to storage and linked to the call record.

---

4. The Turn Loop — the Heart of the Pipeline

A "turn" = the customer says something, we reply. Here is what actually happens inside one turn:

1. Listening. Caller's audio flows continuously: provider → us → STT. STT sends back partial transcripts ("I want", "I want to book", "I want to book an appointment").

2. Knowing when they've finished. This is trickier than it sounds — humans pause mid-sentence. We combine three signals: the STT's own "final transcript" marker, a silence detector (VAD — voice activity detection) watching the audio energy, and a minimum-silence timer (~600–800 ms, tunable). Only when the signals agree do we declare: turn complete.

3. Thinking. The final text goes to the brain: `POST /conversations/{id}/turns`. While the brain thinks, the caller hears silence — this is the window we fight to keep short (Section 5).

4. Speaking. The brain's reply comes back split into sentences (`response_segments`). We push sentence 1 into TTS immediately and start playing its audio while sentences 2 and 3 are still being synthesized. The caller starts hearing the answer as early as possible.

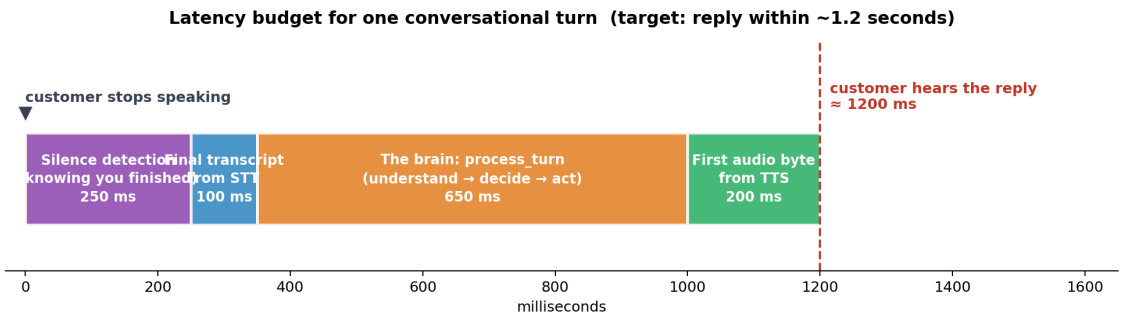
5. Back to listening. The moment we finish speaking (or get interrupted — Section 6), we're listening again.

The session object tracks which phase the call is in at every moment: `listening → thinking → speaking → listening...`. Every phase change is timestamped, which is how we measure latency per turn in production.

---

5. The One-Second Problem

In human conversation, a reply that takes longer than ~1.5 seconds feels broken — people say "hello? are you there?". This is the single hardest engineering constraint in the project.



Our budget per turn (from the customer's last word to the first sound of our reply):

Stage	Budget	Notes
Detecting they've finished speaking	~250 ms	The silence timer is most of this — it's a *deliberate* wait, tunable
Final transcript from STT	~100 ms	Deepgram streaming is fast; this is mostly network
The brain (`process_turn`)	~650 ms	Understand + decide + act + compose (2–3 LLM calls)
First audio byte from TTS	~200 ms	Streaming TTS, first sentence only

Total	**~1.2 s**	Acceptable. Anything over ~2 s needs the tricks below

The tricks that keep us under budget:

1. **Stream everything.** Never wait for a full anything — partial transcripts flow while the customer talks; TTS starts on sentence 1 while the brain may still be finalizing sentence 3.
2. **Start thinking early (speculative start).** When the partial transcript has been stable for a moment and VAD says the voice has stopped, we send the turn to the brain *before* the full silence timer expires. If the customer resumes talking, we throw the speculative result away. Saves 300–500 ms on most turns.
3. **Acknowledgment fillers.** For turns we know will be slow (a tool doing real work — booking, searching), the brain flags it, and the voice layer immediately plays a short natural filler — "Sure, one moment…" — from pre-generated audio, while the real work happens behind it. The perceived wait drops to near zero.
4. **Fast model for understanding.** Step 2 of the planner uses a small, fast model; the big model is reserved for deciding and composing.
5. **Pre-generated audio for fixed phrases.** Disclosure, greetings, fillers, "goodbye" — all cached audio files per tenant persona. Zero synthesis time.
6. **Region matters.** The voice service is deployed in the same Azure region as the telephony provider's India endpoints; STT/TTS providers chosen with India POPs. Every 100 ms of round-trip matters when you cross it 3 times per turn.

Measurement is built in: every turn records per-stage timings into the decision trace. The dashboard's agent analytics show p50/p95 latency per stage — so when the number creeps up, we know exactly which stage to blame.

---

6. Interruptions (Barge-In)

Real customers interrupt: we're mid-pitch and they say "actually, how much does it cost?"

What must happen — instantly:

1. VAD detects the caller's voice while we are in the `speaking` phase.
2. We stop our audio playback **immediately** (target: within ~150 ms — anything slower feels like the agent is talking over them).
3. Whatever remained unspoken of our reply is discarded, but we note *what was actually said out loud* versus what wasn't — the brain needs to know the customer never heard sentence 3.
4. We switch to `listening` and process their interruption as a normal new turn. The note about the unfinished reply goes into the turn metadata so the brain has honest context.

One guard: brief noises (a cough, "hmm", background TV) shouldn't kill our sentence. The interruption only triggers if the detected speech lasts longer than a threshold (~300 ms of sustained voice) or STT immediately produces words. Below that, we keep talking.

---

7. Silence and Awkward Moments

Customer goes quiet mid-call. After ~6 seconds of silence in the `listening` phase: a gentle prompt ("Are you still there?"). After a second timeout: "Seems like this isn't a good time — I'll call you back later. Have a great day," → polite hangup → the brain records it and schedules a callback per rules. Never an abrupt dead-line hangup.

Customer speaks but STT produces garbage (heavy noise, bad line). The STT confidence score arrives with every transcript. Below a threshold, the pipeline doesn't bother the brain — it asks directly: "Sorry, I couldn't hear that clearly — could you repeat?" Two failures in a row → the brain is told, and it typically offers a callback on a better line.

Both talk at once (cross-talk). Barge-in rules win: customer speaking always takes priority; we stop.

DTMF (keypad presses). Detected by the provider and forwarded to us as events — supported for things like "press 1 to confirm" as an accessibility fallback, though the primary interface is always natural speech.

---

8. Recording, Disclosure & Privacy in the Pipeline

- Disclosure **always** plays before recording starts and before any conversation happens. It's a hard sequence in the session state machine, not a convention — the code cannot reach `listening` without passing `disclosure_played`.
- Recordings upload to object storage encrypted, linked to the call record; access is only via the short-lived signed URLs from the API, and every access is logged.
- Live transcripts flowing through the system are PII-masked before they land in any log file (masking design is in the Compliance doc). The database stores the true transcript; *logs* never do.
- If the tenant's policy or the customer's region forbids recording, the session runs with recording disabled — the pipeline works identically, just without the tap.

---

9. Transfers to Humans (Warm Transfer)

When the brain returns `escalation: { type: "immediate" }`:

1. The agent tells the customer honestly: "Let me connect you with my colleague who can help you better. One moment."
2. Comfort audio (soft hold music / "connecting you now") plays.
3. Via the provider's API, we dial the human agent's number/extension and bridge the two call legs.
4. Simultaneously, the human's dashboard pops the **briefing**: who the customer is, journey stage, what happened this call, what's pending — generated by the brain at escalation time.
5. If no human answers within N seconds (configurable), fallback per rules: "All our team members are busy — I've scheduled a callback for you at 4 PM today." The escalation record stays open either way.
6. Once bridged, our AI leg goes silent and drops off; recording continues per policy.

---

10. When Components Fail Mid-Call

Failure	What the caller experiences	What actually happens
STT provider goes down	A brief "sorry, could you repeat that?"	Circuit breaker flips to the fallback STT within one turn; ops alert fires
TTS provider goes down	Slightly different voice mid-call	Same — fallback TTS takes over; better a voice change than silence
The brain times out (>3 s)	"Give me just a second…" then a real answer, or a graceful exit	One retry; if still failing: apology + "I'll call you right back" + automatic callback + alert
Voice service instance crashes	Call drops	Provider webhook tells us the call died; the conversation is closed as `abandoned`; post-call pipeline still runs on whatever transcript exists; customer gets a callback
Media WebSocket hiccups	A moment of silence	Auto-reconnect with jitter buffer; sessions tolerate ~2 s of stream gap

The principle everywhere: **never leave the customer in dead air without an explanation, and never lose the record of what happened.**

---

11. Scaling & Deployment Notes

- One voice service process comfortably handles **~50–80 concurrent calls** (it's I/O-bound — mostly shuffling audio bytes); we scale horizontally by adding instances behind the load balancer. Calls are sticky to their instance (the session lives in process memory, backed by Redis snapshots for crash recovery).
- Audio format on the wire is 8 kHz μ-law (telephone standard) — we transcode to/from 16 kHz PCM for STT/TTS at the edge of the service.
- Concurrency limits per tenant (max simultaneous calls) are enforced here, feeding back into campaign pacing.
- Load testing before launch: synthetic calls with recorded audio at 2× expected peak, measuring the per-stage latency distributions — the pass criterion is p95 under 2 s end-to-end.

Compliance, Security & Audit Design

Compliance, Security & Audit Design

---

1. What This Document Covers

An AI that autonomously calls people and handles their personal data must be provably trustworthy — not "we believe it behaves," but "here is the record showing it behaved." This document covers the five pillars of that:

1. **Consent & Do-Not-Contact** — never contacting anyone we shouldn't
2. **Recording & disclosure** — doing the legally required things on every call
3. **Personal data protection (PII)** — who can see what, and how data is guarded
4. **Access control (RBAC)** — humans and the AI both operate on least privilege
5. **The audit trail** — every decision explainable after the fact

A theme you'll notice throughout: wherever possible, compliance is **enforced by structure** (the code physically cannot do the wrong thing) rather than by policy (the code is asked nicely). Structural guarantees are called out explicitly below.

---

2. Consent Management

What we track

Consent is not one yes/no — we track it per purpose, in an **append-only ledger** (`consent_records`). Nothing is ever edited or deleted; a change of mind is a *new entry*:

Consent type	Covers
`calls`	May we call this person at all
`recording`	May we record the call
`marketing`	May we make promotional/sales calls (vs. only service calls like reminders)
`data_processing`	May we store and process their information

Each entry stores: granted or revoked, when, through which channel, and **evidence** — for verbal consent, a pointer to the exact clip in the call recording where they said yes. If a

regulator ever asks "prove this person consented," the answer is a database row pointing at audio.

How consent is captured

- **Imported leads** carry consent metadata from the source (the client asserts it; we record the assertion and its origin).
- **During calls**, the agent can ask for consent when needed, and a high-confidence "yes" writes a ledger entry referencing that moment in the recording.
- **Revocation is instant and structural:** "stop calling me" triggers the DNC signal (below), which no rule, campaign, or human enthusiasm can override.

---

3. Do-Not-Contact (DNC)

The single most important compliance behavior: **a person who opted out is never contacted again.**

Enforcement points — four layers deep

1. **At dial time (structural).** Every outbound call — campaign, retry, reminder, callback, or manually triggered — passes the pre-call rule check, and the DNC rule is `priority: 0, terminal: true`: it runs first, and if it matches, evaluation stops. There is no code path to the dialer that skips this check.
2. **At campaign creation.** The audience preview already excludes DNC numbers, so the humans planning a campaign see the truth.
3. **In-call.** If a customer says "don't call me again," the DNC signal fires: the customer's stage moves to `do_not_contact` (a terminal stage — no transitions out except by an admin with a logged reason), their number goes on the DNC list *immediately, mid-call*, and the agent confirms it to them: "Understood — I've removed you from our calling list. You won't hear from us again."
4. **On the state machine itself.** The `do_not_contact` transition is the only transition defined with **no guard** — no rule can block it, no confidence threshold applies. Opting out always works.

The list itself

- Scoped entries: `all` (nothing ever) vs. `marketing` (promotional blocked, but service calls like "your appointment is tomorrow" allowed — a distinction Indian telecom rules recognize).
- Sources tracked: customer request, regulator list import (e.g., national DNC registry), manual entry.
- Matching is on the normalized phone number — a customer with two records still can't be called on that number.
- **Removal from the list is admin-only**, requires a typed reason, and is itself an audit event. Additions are one click; removals are deliberately heavy.

---

4. Recording & Disclosure

- The disclosure ("this call may be recorded…") plays **before** recording starts and before any conversation — enforced by the voice session's internal state machine, which cannot reach the listening phase without passing through `disclosure_played` (structural). The flag is stored on every call record.
- Per-tenant and per-region configuration: disclosure text, whether recording is on at all, and retention period. A tenant operating where recording requires explicit verbal consent can enable a consent-question flow before recording begins.
- Recordings are encrypted in storage, playable only through short-lived signed URLs issued by the API, and **every playback is logged** — who listened to which call, when.

---

5. Protecting Personal Data (PII)

The three places data lives, and the rule for each

Where	Rule
The database	The true data lives here — encrypted at rest (disk-level + Postgres), reachable only through the API with role checks. This is the one place real PII is allowed to be.
Log files	**Never contain raw PII (structural).** All logging goes through one wrapper that masks before writing: phone numbers become `+91-98XXXXXX21`, emails `r***l@adv***.com`, and transcripts are logged only as IDs, never as text. Developers cannot accidentally log a phone number, because the logger won't let them.
LLM providers	Prompts are built from context blocks, and the builder masks direct identifiers that the model doesn't need: the model sees "the customer" and their preferences, not their phone number or email. What the model genuinely needs to converse (first name, city, product interests) is sent under our data-processing agreements with the providers. Providers are configured for zero data retention where offered.

Additional measures

- **Encryption in transit** everywhere: TLS on every external and internal hop, including the media WebSockets.
- **Field-level encryption** for the most sensitive columns (beyond disk encryption), keys in Azure Key Vault, rotated on schedule.

- **Data retention:** per-tenant policy — recordings kept N months, transcripts M months, then automatically purged by a scheduled job that logs what it deleted. Aligned with India's DPDP Act principle of not keeping data longer than needed.
- **Right to erasure:** a customer deletion request runs a documented pipeline: personal fields anonymized, recordings deleted, facts purged — while the anonymized interaction skeleton (needed for aggregate analytics and legal defense) is retained. The deletion itself is an audit event.

---

6. Access Control (RBAC)

Human roles

Role	Can	Cannot
`agent`	View customers, calls, appointments; handle escalations; create tickets	Change rules, tools, or configuration; export data; hear recordings by default
`supervisor`	Everything agents can + listen to recordings, correct AI memory facts, manually move customer stages, manage escalation queue	Change engine configuration; manage users
`admin`	Everything: rules, tools, prompts, packs, users, DNC removals, API keys	— (but every admin action is logged, see §7)
`analyst`	Read-only analytics and audit views	See raw PII (their views get masked fields), change anything

Permissions are granular strings (`customers:read`, `rules:write`, `recordings:listen`…) grouped into these roles; custom roles are possible per tenant. Every API endpoint declares its required permission — the check is in one middleware, not scattered through handlers.

The AI's own permissions

The AI is also a permission-holding actor, deliberately *less* privileged than an admin:

- Each tool declares its `permission_level` — most are `agent`-level; destructive or bulk operations can be marked `supervisor_only` or `human_only`, making them **invisible to the AI entirely** (they never enter the candidate list — structural).
- The AI cannot: delete customers, remove DNC entries, edit rules or its own prompts, access other tenants, or export data. Not "instructed not to" — there is no tool for it.

API keys

Service keys are scoped ("this key can read customers and post turns, nothing else"), expire, and are shown once at creation (only a hash is stored). Per-key rate limits.

---

7. The Audit Trail

Two complementary records answer two different questions:

"Why did the AI do that?" — decision traces

Every conversational turn writes one `decision_traces` row containing the complete picture: what the customer said, which memory entries and knowledge chunks were consulted (by ID — reproducible), every rule evaluated and its verdict, the AI's proposed action and stated reasoning, validation outcomes, the confidence score and band, what was ultimately done, which prompt versions and model produced it, and per-stage latency.

This makes the PRD's explainability requirements literal one-query lookups:

Question	Answer source
Why was this tool selected?	`planner_reasoning` + the candidate list in the trace
Why was an action blocked?	`rules_evaluated` — the exact rule, with its reason string
Which memories influenced this?	`retrieved_memory` — fact IDs consulted that turn
Which knowledge was used?	`retrieved_chunks` — chunk IDs + relevance scores
Would this happen again?	Re-run with the same prompt versions (hashed in the trace)

The dashboard renders a trace as a readable timeline — a supervisor clicks any turn of any call and reads the full story. No engineer, no log-diving.

"Who changed the system?" — admin audit events

Every configuration change writes an `audit_events` row: actor, action, resource, **before and after values**, IP, timestamp. Covers: rule edits and activations, tool enable/disable, prompt changes, user/role changes, DNC removals, recording access, data exports, manual stage overrides, memory-fact corrections.

Audit rows are append-only — no API exists to edit or delete them (structural). Retention: minimum 3 years or per contract.

Standing compliance reports

Built as saved queries over these two tables: all blocked actions in a period (and by which rule) · all DNC additions/removals · all recording accesses · all escalations with reasons · hallucination-flag incidents. Exportable for a client's compliance team.

---

8. Platform Security Basics

- **Webhooks** from telephony providers are verified by provider signature (each adapter implements its provider's scheme); unsigned or invalid payloads are rejected and alerted.
- **Rate limiting** per key/user/IP at the gateway; login endpoints get strict limits + lockout with backoff.
- **Secrets** (provider API keys, DB credentials, signing keys) live in Azure Key Vault, injected at runtime — never in code, images, or environment files in the repo.
- **Network posture:** only the API gateway and the telephony webhook/media endpoints are public. Database, Redis, and workers are on a private network. SSH by key only, no password auth.
- **Dependencies** scanned in CI (pip-audit / npm audit); base images pinned and rebuilt on schedule.
- **Tenant isolation** is enforced twice: every repository query is tenant-filtered (application layer), *and* Postgres row-level security is enabled on the sensitive tables as a second net — a bug in the first layer still hits the second (structural, defense in depth).
- **Backups:** automated nightly, encrypted, restore-tested monthly (a backup that was never restored is a hope, not a backup).

---

9. Regulatory Mapping (India-first)

Requirement	Where it's handled
TRAI/TCCCPR telemarketing rules — DND registry, permitted calling hours (typically 9 AM–9 PM), promotional vs. transactional distinction	DNC list with scopes (§3) + `calling_policy` rules; national registry importable via `/dnc/import`
Recording disclosure	§4, structural
DPDP Act 2023 — consent, purpose limitation, retention limits, erasure rights	Consent ledger (§2), retention jobs + deletion pipeline (§5)
Sector rules for specific clients (e.g., IRDAI for insurance, RBI adjacency for lending)	Arrive as pack-level rule sets and disclosure scripts — the platform mechanism is already generic

The platform ships the *mechanisms*; each tenant's pack + rule configuration expresses their specific obligations. New regulation → new rules, not new code.

---

10. Escalation as a Compliance Feature

The human-handover system (Core Engine doc §7) is also the compliance pressure valve, so it's worth restating from this angle: legal threats, payment disputes, harassment claims, or a request to speak to a human **always** route to a person — these triggers live in the always-on `escalation_policy` rule set that packs can extend but not remove. And once escalated, automated contact with that customer is locked until a human closes the loop. An AI that knows when to stop is the strongest compliance feature of all.

Deployment & Testing Strategy

Deployment & Testing Strategy

---

1. What This Document Covers

Two questions every serious project must answer before writing much code:

1. **How does this run?** — environments, servers, deployment process, monitoring, and what we do when something breaks at 2 AM.
2. **How do we know it works?** — and for an AI agent, "works" means more than passing unit tests: it means the agent *behaves correctly in conversation*, which needs its own kind of testing.

---

2. Environments

Environment	Where	Purpose	Telephony
Local	Each developer's machine (Docker Compose)	Development. Full stack runs locally except real calls	Simulated calls (see §7.3)
Staging	Small Azure VM	Pre-release testing, client demos	Real provider account with test numbers — real calls to team phones
Production	Azure VM(s), same region as telephony provider's India endpoints	Live	Real

Rule one: **staging is identical to production in shape** — same Docker images, same compose layout, smaller machine. Anything that only exists in production is a bug waiting for its moment.

Rule two: staging uses a separate database and separate provider accounts. No shared anything.

---

3. Production Deployment Layout

Everything is containerized and orchestrated with **Docker Compose** (your current PM2-on-VM approach works for Node-style processes, but here we have Python services, workers, Redis, and Postgres with exact version needs — containers keep all of it reproducible; and it's one `docker compose up -d` instead of a hand-configured VM).

```
Azure Ubuntu VM (production)
├── caddy                # reverse proxy + automatic HTTPS
├── core-api       ×2    # the Core Platform (two replicas)
├── voice-service  ×2    # the Voice Session Service
├── worker         ×2    # Celery workers (post-call, campaigns, retries)
├── beat           ×1    # the scheduler (reminders, retention jobs)
├── dashboard            # React build, served as static files by caddy
├── postgres:16          # with pgvector; nightly encrypted backups → Azure Blob
└── redis:7              # persistence on (AOF)
```

Sizing to start: one `Standard_D4s` (4 vCPU / 16 GB) carries the whole stack comfortably at launch scale (tens of concurrent calls). The first scaling move, when needed, is not Kubernetes — it's: move Postgres to Azure's managed Flexible Server, and put the voice service on its own VM (it's the component whose load grows with concurrent calls). That two-step gets you to hundreds of concurrent calls before any orchestration complexity is justified.

Configuration & secrets: all config via environment variables (pydantic-settings); secrets pulled from **Azure Key Vault** at container start — never stored in the repo, images, or a `.env` on the server.

Database migrations: Alembic, run as an explicit deploy step (not on container boot). Migrations must be backward-compatible one version back (add column → deploy code → later remove old column), so a code rollback never requires a database rollback.

---

4. CI/CD Pipeline (GitHub Actions)

Every push to a pull request runs, in order:

1. **Lint + type check** (ruff, mypy) — cheap failures first
2. **Unit tests** — fast, no external services
3. **Integration tests** — against real Postgres + Redis spun up in the pipeline
4. **Engine behavior suite** — the conversation tests from §7.2 (the ones that don't need paid LLM calls run on every PR; the full suite runs nightly)
5. **Build images** — tagged with the git SHA

Merge to `main` → auto-deploy to **staging**. Production deploys are **manual approval** (one click, but a human clicks it), deploying the exact staging-verified image by SHA — we never rebuild for production.

Rollback is redeploying the previous SHA: one command, under two minutes. Every deploy is recorded (who, what SHA, when) in the audit events.

---

5. Monitoring — Knowing It's Healthy

Three layers, from cheap to deep:

1. Is it up? `GET /health/deep` checked every 30 s by an external uptime monitor (the VM cannot monitor itself). Checks DB, Redis, and each provider's reachability.

2. Is it performing? Prometheus + Grafana dashboards on the numbers that matter:

- Per-stage turn latency (p50/p95) — the §5 voice budget, live
- Concurrent calls, call setup failures, provider error rates and circuit-breaker state
- Queue depths (a growing Celery queue means the post-call pipeline is falling behind)
- LLM token spend per hour (cost surprises show up here first)

3. Is it behaving? The agent-quality numbers, computed from decision traces:

- Escalation rate, blocked-action rate, average confidence
- Hallucination flags (grounding checks that failed in step 11)
- Conversation outcomes distribution

Alerts that page a human (kept deliberately short so alerts stay meaningful): API down · p95 turn latency > 2.5 s for 5 min · provider circuit breaker open · queue depth beyond threshold · disk > 85% · **any hallucination flag** (rare enough to investigate individually) · error rate spike.

Logs: structured JSON (structlog), PII-masked at the logger (Compliance doc §5), shipped to one place, searchable by `request_id`/`trace_id` — the same IDs the API returns in errors, so a bug report can be traced end to end in minutes.

---

6. Cost Watch (worth its own section)

Voice AI has a per-minute marginal cost (telephony + STT + TTS + LLM ≈ ₹4–9/min depending on providers and call complexity). Two protections from day one:

- A **per-tenant daily budget** in config; crossing 80% alerts, crossing 100% pauses outbound campaigns (inbound and service calls continue).
- The cost per call is computed and stored on each call record — so "which campaign / which conversation type is expensive" is a dashboard query, not a spreadsheet archaeology project at month-end.

---

7. Testing Strategy

The pyramid, from bottom (many, fast, cheap) to top (few, slow, real):

7.1 Unit & integration tests (the classical base)

Unit tests for all deterministic logic — and note that our safety-critical code is *deliberately* deterministic, which makes it thoroughly unit-testable: rule evaluation, tool validation checks, slot computation, state transitions, confidence banding, PII masking. Target: this layer alone catches any regression in the safety machinery.

Integration tests against real Postgres/Redis for: repositories with tenant isolation (including proving that tenant A queries can't see tenant B rows), the booking transaction under concurrent access (two simultaneous bookings of the last slot — exactly one must win), migration up/down, webhook signature verification.

7.2 Engine behavior tests (the layer specific to AI agents)

This is where an agent project lives or dies. We maintain a **conversation test suite**: scripted dialogues with assertions, run against the real engine with either a mocked LLM (deterministic, every PR) or the real LLM (nightly + pre-release).

```
# tests/conversations/booking_happy_path.yaml
setup:
  customer: { state: interested, facts: [{ category: preference, key: preferred_time, value:
morning }] }
  scheduling: { centre: vapi_main, free_slots: ["2026-07-18T10:30", "2026-07-18T11:30"] }
turns:
  - user: "I'd like to come in this Saturday"
    expect:
      action_in: [respond, confirm]
      must_mention_only_slots_from: scheduling.free_slots   # anti-hallucination assertion
  - user: "10:30 works"
    expect: { action: confirm }                          # requires_confirmation gate
  - user: "yes, confirm it"
    expect:
      tool_executed: book_appointment
      state_changed_to: appointment_booked
```

The suite covers, at minimum: every tool's happy path · every `failure_policy` branch (slot taken, system error) · every rule (DNC mid-call, calling hours, identity verification demanded) · confidence behavior (garbled input → clarify, not act) · escalation triggers (angry VIP, legal keywords) · the interrupting states (call-later then resume) · and **adversarial turns**: the customer asks for a discount that doesn't exist, claims a booking that doesn't exist, tries to change someone else's appointment. Every incident found in production later becomes a new test here permanently — the suite only grows.

Rule tests specifically: the `POST /rules/{name}/test` dry-run endpoint is also used in CI — each rule set ships with scenario fixtures asserting exactly which rules fire.

7.3 Voice pipeline tests

- **Simulated calls:** a test harness plays recorded audio files (clean speech, noisy speech, interruptions, silence, Hinglish accents) through the full pipeline as if from the provider — runs in CI, no phone network needed. Asserts transcripts, barge-in stop time (<150 ms), silence-prompt behavior, disclosure-before-recording ordering.
- **Real-call tests (staging, scripted, manual):** a checklist of ~25 scenarios executed by the team on real phones over the real provider before every release: booking end-to-end, interrupting mid-pitch, going silent, saying "stop calling me", asking for a human, poor-network call from a mobile. Each scenario has pass criteria including subjective ones ("did the pause feel natural?") — voice quality cannot be fully asserted by machines.

7.4 Load testing

Before launch and before any big campaign: synthetic concurrent calls (the simulated-call harness, parallelized) at **2× expected peak**, sustained 30 minutes. Pass criteria: p95 end-to-end turn latency < 2 s · zero dropped sessions · zero double-bookings under concurrent slot pressure · queue depths recover after the wave.

7.5 The golden set (regression protection for the AI's judgment)

~100 real (anonymized) conversation turns with human-approved expected behavior, re-run against the engine whenever anything that shapes decisions changes: prompts, models, provider swap, retrieval tuning. The diff report — "the new prompt changed the agent's action on 4 of 100 goldens, here they are" — is reviewed by a human before activation. This is what makes "let's improve the prompt" safe instead of a gamble.

---

8. Release Process

1. Feature merged → auto-deployed to staging
2. CI green including nightly engine suite with real LLM
3. Real-call checklist on staging (§7.3) — for releases touching voice
4. Golden-set diff reviewed — for releases touching prompts/models/retrieval
5. Manual approve → production deploy (same image SHA)
6. **Canary window:** first 30 minutes watched live on the dashboard; one command rolls back
7. Config changes (rules, prompts, packs) don't need deploys at all — they're versioned rows with one-click rollback (their own release process is: edit → dry-run test → activate → watch)

---

9. Go-Live Checklist (per tenant)

Infrastructure ready is not launch ready. Before a tenant's number goes live:

- [ ] Industry pack loaded; rules dry-run tested against scenario fixtures
- [ ] Knowledge base uploaded; retrieval spot-checked with `POST /knowledge/search` on the 20 most likely customer questions
- [ ] DNC registry imported; calling-hours rules match the tenant's region
- [ ] Disclosure script legally reviewed by the tenant
- [ ] Escalation path staffed — a real human answers the transfer number during calling hours (test it)
- [ ] Provider numbers verified, webhook signatures confirmed, budgets and alerts configured
- [ ] Real-call checklist executed on the tenant's own configuration
- [ ] The tenant has dashboard access and knows how to: pause everything (one button), read a decision trace, and edit a rule
- [ ] First campaign starts small: 50 calls, human reviews outcomes and a sample of traces, then scale

---

10. When Things Break — the Short Runbook

Symptom	First response
Latency alert	Grafana → which stage grew? Provider stage → check circuit breaker/fallback. Brain stage → check LLM provider status + token queue
Calls failing to connect	Provider dashboard + webhook log; verify signature failures aren't rejecting events
Queue depth climbing	Scale workers (`docker compose up -d --scale worker=4`); investigate the slow task after
Hallucination flag	Pull the trace, find which grounding check failed, add the case to the conversation suite, fix, redeploy
Bad rule activated	One-click rollback to previous version in the dashboard — no deploy
Full outage	Redeploy previous SHA; if data-related, restore drill procedure (practiced monthly, §8 of Compliance doc)

Every incident ends with a new test in the suite. That's the loop that makes the system *more* reliable every time it fails.

Project Timeline

Project Timeline

AI Voice Sales & Customer Lifecycle Agent — 4 Weeks

---

1. Summary

Duration	4 weeks
Team structure	Two parallel tracks: Backend & Voice, and Frontend
Delivery goal	Working system: real phone calls conducting complete conversations — customer identification, product Q&A from the knowledge base, appointment booking with confirmation, ticket creation, and post-call memory updates — with the operational dashboard
Key milestone	End of Week 4 — a real phone call books a real appointment end-to-end, demonstrated live
Deferred work	CI/CD, full test suites, production deployment hardening, and selected advanced features

---

2. Track Plan — Week by Week

Backend & Voice Track

Week	Focus	Deliverables

1	Foundation	Repository and Docker setup; full database schema with migrations; auth and RBAC framework; frozen `process_turn` contract; CRM services and APIs; customer journey state machine; telephony provider account with first live test calls (early latency measurement)
2	Engine core	Planner loop; tool registry and validation; core tools (book / reschedule / cancel appointment, create ticket, CRM update); essential rules (Do-Not-Contact, calling hours, identity verification); scheduling engine with transactional booking; confidence gating
3	Text milestone → Voice	Complete conversation working over text (engine verified end-to-end); then voice pipeline: streaming STT and TTS integration, turn detection, recording with disclosure sequence, basic interruption handling
4	Integration & knowledge	Voice service connected to engine; **real call books real appointment**; knowledge base ingestion and retrieval (RAG); post-call summarization and memory update; demonstration preparation

Frontend Track

Week	Focus	Deliverables
1	Foundation	React + Vite + Tailwind scaffold; routing and layout; component library (tables, forms, modals, states); login and role-based navigation
2	Core screens	Customer list with search and filters; customer profile with facts and timeline; appointment and slot screens — built against the API specification with mocked responses, switched to live CRM and scheduling APIs at the Week 2 integration point

3	Operational screens	Call log with detail view; conversation transcript viewer; knowledge base upload and management; integration with live call and knowledge APIs
4	Completion	Ticket screens; rules view (read-only); recording playback; screen polish and end-to-end walkthrough as part of the final demonstration

Integration Points

End of week	Integration
2	Frontend customer and appointment screens switch from mocked to live APIs
3	Frontend call and knowledge screens live; engine verified over text
4	Full system: live call visible end-to-end — conversation, booking, dashboard, memory

---

4. Milestones

End of week	Milestone
1	Infrastructure and schema live; test calls completed through provider; dashboard foundation ready

2	Booking works via API with all safety checks; customer screens live on real data
3	Complete conversation over text; voice pipeline operational
4	**Live demonstration: real phone call → conversation → booked appointment → visible in dashboard → memory updated**